

In Case You Need Cleaning

An interactive development project exploring AI-supported automation in service booking.

Elia H. Ullerichs, Gioia Sciommeri, Luna A. L. Jensen, Mads R. Frederiksen, Mathias W. Albjerg, Sofie L. Fuglsang Group 3

22/12/2025

DAD9-P9



Title: In Case You Need Cleaning

An interactive development project exploring AI-supported automation in service booking.

Theme: Development of Interactive Systems

Projektperiod: 2nd of September 2025 -
22nd of December 2025



AALBORG UNIVERSITET
STUDENTERRAPPORT

Project Group 2

Participants:

Elia H. Ullerichs

Elia H. Ullerichs
Study No. 20212834

Gioia Sciommeri

Gioia Sciommeri
Study No. 20242997

Luna A. L. Jensen

Luna A. L. Jensen
Study No. 20213072

Mads R. Frederiksen

Mads R. Frederiksen
Study No. 20242494

Mathias W. Albjerg

Mathias W. Albjerg
Study No. 20213001

Sofie L. Fuglsang

Sofie L. Fuglsang
Study No. 20212701

Supervisor: Ilhan Aslan

Institut: Institut for Datalogi

Hand-in date: 22. December 2025

Abstract

This project investigates the potential and limitations of automating manual appointment coordination for the hypothetical case of Unit Cleaning, a service company challenged by administrative tasks. To address these challenges, the project follows an iterative Agile Software Engineering approach, moving from an initial prototype focused on basic digitisation to a second prototype centred on supportive Agentic Artificial Intelligence.

The resulting system uses a Flask-based backend with a custom AI-agent capable of performing scheduling actions and providing recommendations based on user information, weather data, and calendar availability through conversational interaction. An evaluation based on Jakob Nielsen's usability heuristics shows that while the AI-agent can reduce administrative effort, it also introduces challenges related to user trust, system transparency, and cognitive load. The project concludes that effective conversational automation is best supported by a hybrid interface, which balances the flexibility of an AI-agent with structured visual elements to ensure the user remains in control of the automated process.

Character count: ~110.000

Pages: 51

Nr. of appendixes: 5

The content of the report is freely accessible, but publication (with source attribution) may only take place by agreement with the authors.

Contents

1	Project Trajectory	1
1.1	Work process	1
2	Organisational Problem Domain	4
2.1	State of the art	4
2.1.1	Why this Project Matters	5
2.2	Unit Cleaning Case Study	6
2.3	Problem statement	9
3	First rendition	10
3.1	Our First Prototype	10
3.1.1	Unfolding the problem domain	10
3.2	Implementation of the first prototype	12
3.3	Insights From the First Rendition	14
4	Second Prototype	15
4.1	Defining the Problem Domain	15
4.1.1	User story deliveries	16
4.1.2	Users and Agentic AI	18
4.1.3	Problem domain summary	19
5	Application domain	21
5.1	Design choices	21
5.1.1	Use case	22
5.1.2	Requirements	24
5.2	Implementation of the second prototype	25
5.2.1	Architecture of system	25
5.2.2	Backend	27
5.2.3	AI	31
5.2.4	Frontend	34
5.2.5	Subconclusion	37

6	Evaluation & Reflection	38
6.1	Case partnership feedback	38
6.2	Heuristic Evaluation	39
6.2.1	How we used the Method	39
6.2.2	Identified Usability Issues	40
6.2.3	Prioritising Heuristics	43
6.2.4	Justifying the chosen heuristics	44
6.2.5	Suggestions for improvements	45
6.3	Reflections on the Findings	47
6.3.1	Balancing Automation and Cognitive Effort	47
6.3.2	Efficiency, reliability and User Trust	48
7	Conclusion	51
	Bibliografi	52

Abbreviations

- **AAI**: Agentic Artificial Intelligence
AAI & AI-agent is interchangeable
- **AI**: Artificial Intelligence
- **API**: Application Programming Interface
- **APM**: Agile project management
- **ASE**: Agile Software Engineering
- **CRUD**: Create, Read, Update, Delete
- **CUI**: Conversational User Interface
- **ER**: Entity Relationship
- **GUI**: Graphical User Interface
- **HE**: Heuristic Evaluation
- **IDE**: Integrated Development Environment
- **LLM**: Large Language Model
- **MCP**: Model Context Protocol
- **OOAD**: Object-Oriented Analysis & Design
- **RAG**: Retrieval-Augmented Generation
- **RM**: Risk Management
- **SMTP**: Simple Mail Transfer Protocol
- **UCD**: Use Case diagram
- **UI**: User Interface
- **UX**: User eXperience
- **XP**: eXtreme Programming

Preface

We present this project, offering an in-depth look at the iterative process from initial case to final product. We would like to thank our project supervisor, Ilhan, for his support and valuable feedback. We also thank our case collaboration partners, as well as all participants who contributed their time and insights during the development and evaluation of the prototype.

Generative AI as a tool

Throughout the project, generative AI was used as a supportive tool in both the development process and the writing of this report. When working on the written content, AI was used to help with grammar, clarity, and structuring during drafting. All sections were written and revised by the project group, and generated suggestions were never included directly without being critically reviewed and adapted. In the early design phase, generative AI was used to explore initial visual ideas for personas and interface elements. This made it easier to quickly test and discuss different concepts before settling on our final direction. While these early visuals helped inform design decisions, all visual assets used in the final prototype and documentation were created manually by the group using digital drawing tools. Generative AI was also used as support during the technical development of prototypes, particularly when working with elements of the code and debugging. It helped us reason about implementation challenges, consider alternative solutions, and reflect on design choices related to the architecture and AI integration. Using generative AI in this way supported an efficient workflow. It helped us explore ideas, solve problems, and improve our solutions, while the system design, implementation, and analysis presented in this report remain the result strictly of our own work and decisions.

1 Project Trajectory

Our goals this semester

Throughout the semester, we wanted to explore how the methods and tools we have learned in earlier courses would hold up when applied to a concrete organisational practice. Working with the Unit Cleaning case, we stepped into a more realistic setting and investigated how an interactive system might support an existing workflow in practice. We took the chance to work hands-on with analysing practices, identifying where challenges emerge, and experimenting with how interactive systems could support the workflow. Through the process, we reflected on how theory and our prior experience translate into practice, and how interactive systems could create value when implemented in an organisation.

Case Description

For this project, we worked with a case partner operating under the company name Unit Cleaning, who presented a hypothetical case focused on improving how customer information and appointment reminders are handled. The company offers professional cleaning services to private customers and currently manages customer data and scheduled cleaning appointments through manual processes, which makes reminders inconsistent and can lead to missed appointments. Our case partner requested a digital system with a database for managing customer and appointment information, a frontend web application for employees to create, update, and delete records, and an automated notification service that sends personalised reminders the day before a scheduled cleaning. The intended purpose of the system was to improve communication with customers, reduce missed appointments, and support a more efficient and automated scheduling process. The full case description can be found in [Appendix A](#).

1.1 Work process

To meet the goals outlined above, it was necessary to establish a work process that could support both our technical exploration and continuous refinement of a solution. Therefore, throughout the project, we aimed to create a clear structure in our work to prevent the process from becoming unmanageable. During the semester, we had lectures on *Agile Software Engineering* (ASE), which gave us practical frameworks for organising our work and maintaining a stable development flow. Our overall way of working was inspired by the ideas from the Agile Manifesto, which emphasises individuals and interactions over strict processes, working software over extensive documentation, and continuous collaboration and response to change rather than fixed plans. This approach shaped the way we organised and adjusted our work during the project ([Beck et al. 2001a](#)). We selected agile elements from the lectures we deemed to be most fitting for our project, which consisted of applying *Scrum*.

Scrum is a lightweight agile framework used to support the development of complex products through iterative processes, continuous feedback, and close collaboration within a team (Schwaber and Sutherland 2020). It is built on the principles of transparency, inspection, and adaptation and structured development through *Sprints*, a *Product Backlog*, and daily SCRUM meetings (Schwaber and Sutherland 2020). In our project, the *Scrum Master* was responsible for maintaining the *Scrum board* and supporting the structure of the process, while the team was responsible for developing the product within each Sprint (Schwaber and Sutherland 2020, pp. 6–8).



Figure 1.1: Scrumboard

We started the process by using a Project Backlog as our main visual planning tool. This gave us an overview of tasks, responsibilities, priorities, and elements that required approval from the entire group. It made our progress more visible and helped us stay aligned throughout the development process. From that point onward, we worked in iterative cycles of Sprints (Schwaber and Sutherland 2020, pp. 8–11).

In terms of our developmental work, we applied certain elements from *eXtreme Programming* (XP), with focus on having short development iterations (Larman 2004). Rather than relying on fixed plans, we evaluated our progress regularly and adjusted along the way. This made it easier to improve our solutions during the process instead of facing major changes at the end of the two prototypes.

In the technical collaboration, we also used practices that supported structured teamwork. We used *Entity Relationship* (ER) diagrams to model the database structure before implementation (see 5.4), as this made entities and relationships clear and secured a shared understanding of the problem domain (Mathiassen 2018, pp. 6–8). We worked in a shared coding repository, but used individual branches to avoid overwriting each other's work. When a task was finished, it was reviewed as a group and merged into the main branch. This helped remove outdated branches as the project evolved. For more complex tasks, we used pair programming,

which helped us reach shared solutions and understanding faster, and ultimately reduced the risk of errors (Larman 2004).

We also worked actively with *risk management* (RM) to identify tasks that could create challenges and required extra time. According to Pressman 2000, RM is a proactive process where potential risks are identified, analysed, and handled before they develop into actual problems (Pressman 2000, pp. 744–749). In our project, this meant that we discussed the technical challenges, our time pressure, and our coordination risks when they arose, and adjusted our plan accordingly. The importance of balancing flexibility with structured planning based on project risks is also highlighted by Boehm and Turner, who argue that agile and plan-driven approaches should be adapted to the context of the project (Boehm and Turner 2003, pp. 1–2). In the same sense, we worked with quality management in the form of internal regular sprint reviews. These helped ensure that the solutions we delivered worked as intended and followed the decisions made earlier in the project. In agile development, quality is closely connected to continuous evaluation and improvement throughout the process rather than only at the end (Sommerville 2016, pp. 714–716). Overall, our work process was characterised by a balance between both structure and flexibility. By combining Scrum as our main organisational framework with principles inspired by XP in our coding approach, we were able to work iteratively, respond to needed changes, and continuously improve our solutions. At the same time, our use of structured collaboration practices, RM, and quality management contributed to supporting a stable development process.

With the project context, case background, and working methodology established, the report now turns to a deeper examination of the organisational problem domain. This provides the analytical foundation for understanding why the proposed system is relevant and how it addresses existing challenges.

2 Organisational Problem Domain

2.1 State of the art

Current Booking Systems

To understand the challenges faced by Unit Cleaning, it is necessary to look at how similar problems are currently addressed within existing booking platforms in the cleaning service sector. A prominent example of this is HappyHelper, an online service platform that acts as an intermediary between private customers and self-employed helpers. Customers who want cleaning services can create a booking through the platform and are either matched with a nearby helper or allowed to choose a specific cleaner themselves. The helpers working through HappyHelper are relatively independent, as they set their own hourly rates based on their experience and skill level. This means customers can select anything from beginners to highly experienced professionals.

The booking process consists of having the customer enter their address, preferred time, the size of the home and the type of cleaning required. Based on this information, the system automatically estimates how long the cleaning is expected to take. The platform also provides a list of cleaning supplies the customer must have ready, as helpers do not bring their own equipment, they simply show up and perform the cleaning within the estimated time slot. This understanding is based on domain knowledge obtained through dialogue with individuals with practical experience in the platform-based cleaning sector and HappyHelpers own homepage ([HappyHelper 2025](#)).

In recent versions of the system, HappyHelper has introduced several additional features. Customers can now suggest alternative days and time slots, which helpers can choose between. Frequent cancellations negatively affect this rating, making the helper appear less stable and trustworthy. The platform also includes a communication module that allows customers and helpers to chat directly. The module is used to clarify details such as which cleaning supplies are needed or whether they have any special requests. Both parties have access to an overview of upcoming appointments and a contract summary. Reminders are typically sent via email and SMS, while in-app notifications are only partially supported. Despite the digital interface, a significant part of the coordination still happens manually. Changes in appointments, scheduling problems, or technical issues often require contacting HappyHelpers' support team, who handle administrative tasks behind the scenes.

This includes earlier problems where changes to dates or skipped weeks were not correctly registered in the system from the user, and the helper’s view, forcing users to restart the booking process entirely. While HappyHelper is designed to make booking easy for the customer, it has its shortcomings. Its reliance on ratings, manual coordination, and support-driven adjustments shows that much of the operational flow remains dependent on human handling, even though the surface experience appears to be fully digital. These shortcomings reveal a gap between perceived digital convenience and actual operational automation, highlighting the potential need for automation in booking workflows.

2.1.1 Why this Project Matters

Value Creation

This project approaches value creation from the perspective of the organisational challenges described in the Unit Cleaning case (see [Appendix A](#)). Instead of measuring value in abstract terms, we focus on how changes to the booking workflow may affect everyday work practices for both customers and administrative staff. By introducing automation into a manual coordination process, we investigate where interactive systems can meaningfully support users and where limitations could appear. To structure this, we focus on three central areas in which automation has the potential to influence the booking process:

1. **Operational efficiency:** By automating the distribution of time slots and availability, the system drastically reduces the administrative burden. It minimises the time spent on scheduling errors and managing rescheduling, allowing resources to be directed towards the service delivery rather than administration ([Chowdhury 2024](#)).
2. **Service reliability:** Unlike manual workflows prone to human oversights, an automated agent ensures consistent execution. Reminders are sent without errors, and constraints are evaluated with precision. Research has shown that automated reminders significantly reduce the frequency of no-shows ([Gurol-Uganci et al. 2013](#)).
3. **Business scalability:** The automated system separates the administrative workload from business growth. The current setup of the Unit Cleaning proposal indicates that, as the business model grows, there will be an increase in workload for administrative workers. With an AI-agent, the system should be able to handle a significant increase in bookings without an increase in administrative costs.

An increase in efficiency, reliability, and scalability should improve the customer booking experience by reducing friction and waiting time, while also removing the bottleneck currently limiting company expansion. These considerations frame our motivation for this project and clarify why exploring automation through an AI-supported booking system could represent a relevant design direction.

2.2 Unit Cleaning Case Study

Company Description

To work with the challenges described in the case, we start by examining the company and its current way of handling bookings and customer coordination. The case was provided by professional IT developers and entrepreneurs. Unit Cleaning is a hypothetical company that was used as the case study for this report. The company represents a small company operating within the professional house cleaning service sector. The company's operation and structure are derived from problems described by industry experts, as well as second-hand accounts from users of said industry experts. This allows for a focused analysis of a complex real-world problem domain without constraints of a specific company's system or data restrictions. Unit Cleaning provides professional house cleaning services directly to private clients. The current organisation of the company are challenged by scheduling logistics and the customers' experience in handling appointments, as the customer data is handled manually.

Problem Analysis

The problem described by Unit Cleaning lies in the lack of automation across multiple areas of workflow, in particular, the booking and reminder management. The current operation is highly labour-intensive and lacks modern infrastructure. The storage of user data, including names, addresses, and cleaning appointments, is created outside of an integrated database, presumably spreadsheets, digital calendars, or physical documentation. The rich picture in Figure 2.1 shows the problem we identified. The correspondence in regard to scheduling, confirmation, and reminders is handled manually by administrative staff, relying on human consistency and diligence. Fragmented storage and manual processes can result in negative outcomes that align with the need for intervention, mentioned in Section 2.1.1.

Currently, the reliability of the staff to manually handle reminders often leads to inconsistent timing or oversights, resulting in higher rates of missed appointments and no-shows. Adjustments in scheduling require significant administrative effort to manually update records, reducing staff time and resources from core business activities, and becoming more cost-effective. The current setup of the manual system is basically non-scalable as the increase of customers significantly increases the administrative burden, creating a ceiling for the company's potential growth and profitability.

The difficulties at Unit Cleaning show the project's central premise that the core issues of the service industry are rooted in the lack of automation. The need for developing a solution is not only to increase the firm's cost-effectiveness but also enhance the customer's experience, through a reliable and responsive service.

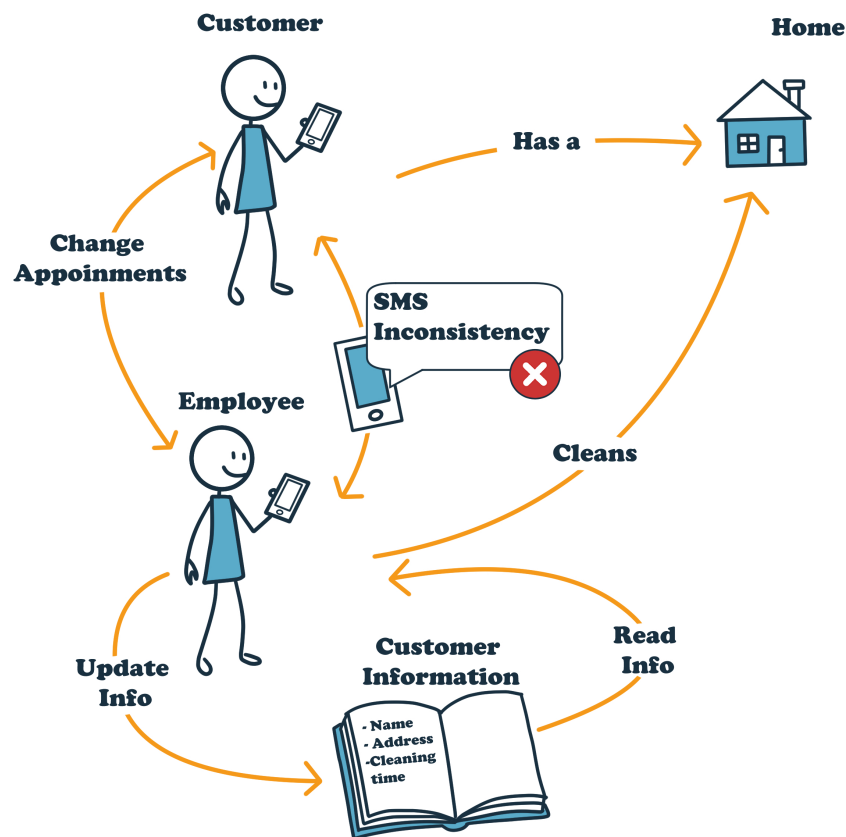


Figure 2.1: Rich Picture of the initial problem described by Unit Cleaning

Refinement of Project Scope

Through discussions with Unit Cleaning, we decided on a few changes to their proposal see section 1.

The first is the shift from staff-centric to customer-centric as the central point for booking requests. This allows customers to handle the main aspect of the booking through the system themselves. This addressed the problem of time dependency, as customers can schedule or manage appointments immediately, outside of administrative working hours.

The second is regarding SMS reminders. The necessity lies in the ability to send stable notifications to the user, reminding them of their appointments. For a better *user eXperience* (UX), the reminder has potential in working beyond a single type of notification, thus allowing different types of notifications to be customised to the individual user. With the focus on automation rather than specific types of notifications, we focus on developing the reminders through email, as it is less costly in the development phase. The intention would then be to implement SMS and other relevant notification types closer to the deployment of the system.

Lastly, the major change is in the concept of automation. The original proposal was a digitalisation of the current service, but as described throughout this section, we have sought to reach further than the original goal set by our case-partners. From this insight, we proposed integrating an AI-agent as a means of achieving a competitive advantage and a high degree of automation. Unit Cleaning showed a strong interest in the prospect of working with *Artificial Intelligence* (AI) to further enhance the customer experience.

The Shift to Conversational and Agentic AI

To address the limitations of the static interface, the future of booking systems could take a shift towards *Agentic Artificial Intelligence* (AAI), AI models that would not just be able to recommend a booking date, but actively perform the booking on behalf of the user (Stryker, n.d.). While traditional *Graphical User Interfaces* (GUI) are effective for direct data entry, they place the burden of coordination on the user, requiring them to manually synthesise availability, preferences, and logical steps. Currently, AAI is used with *Conversational User Interfaces* (CUI), where the user uses the system through natural language prompts rather than GUI components (Jacob Nielsen 2023a; Stryker, n.d.). Through CUIs, our goal is to reduce the administrative burden by automating the coordination process. Instead of users managing the workflow, as seen by platforms like HappyHelper, the AAI handles the complexity. Another benefit of AAI in general is its ability to pass more of a human's cognitive load to the AI-agent. This benefit is demonstrated by the Foo et al. 2022 conference paper 'Paprr Readr Bot'. It shows how conversational agents can successfully reduce the cognitive load, as the coordination logic is moved from the user to an automated computerised logic. Foo et al. 2022 highlights through a "hybrid interface", with a combination of chat widgets and visual data, can move the coordination logic from the user to the AI-agent. In this context, the AI-agent acts like an intermediary, gathering summaries, taking notes, and generating citations, leaving the reader to use this information quickly rather than having to find this information themselves. In the context of booking, this means that the AI-agent acts as the intermediary, connecting data such as calendar appointments with timestamps and addresses, availability in one's schedule, and personal preferences, through a context-based logic.

As a result of the case shifting to a broader scope, as well as our introduction to AAI, we have found it relevant to delve into the following problem statement.

2.3 Problem statement

How can an AI-supported conversational booking system be designed to investigate the potential and limitations of automating appointment coordination in Unit Cleaning's context?

This project explored how an AI-supported booking system could be designed to respond to the coordination challenges described in Unit Cleaning's case. Rather than treating automation as a guaranteed improvement, we focus on understanding the existing booking workflow and identifying where AI could meaningfully support or automate specific tasks. To clarify our intentions, we have extracted these values from the term "automation":

- Reducing the cognitive load on users: Moving the burden of coordination and decision-making from the user to the system.
- Reducing the time one takes to perform a desired task: Streamlining the process so users can reach their goal faster than through manual communication

The problem statement specifically guided the development of the second version of our prototype. The first prototype focused on the needs derived from the case, specifically the digitisation of the notification systems to prevent missing scheduled appointments. With this in mind, we shift the development goal from digitising a manual process to instead building a system able to execute and decide on booking tasks autonomously based on user input and data. The following chapters delve into this process from start to finish, starting off with our first rendition of the project.

3 First rendition

This chapter delves into the development of the first prototype of this project, as well as how it has shaped the following work. This includes fully delving into how we understand the problem, especially regarding digitalising the workflow presented to us in the case description of Section 1. Furthermore, we will delve into the actual implementation of technical aspects, of which the prototype consisted.

3.1 Our First Prototype

The original case presented by our case partners concerned the development of an end-to-end digital system for managing customer information and issuing reminders prior to scheduled cleaning appointments. The first prototype consisted of a database, made to store dummy data for customers and appointment. Meanwhile, a Streamlit-based frontend sought to enable rapid development of an interactive interface which could support basic *Create, Read, Update, Delete* (CRUD) operations. Additionally, the notification module was built to retrieve upcoming appointments, generate personalised reminders, and finally dispatch them.

The intentions of developing this first prototype were to have an initial picture of what could solve the given case, in which the case partners could have an idea of our proposed solution. At this stage, the prototype primarily functioned as a learning tool rather than a finished solution, allowing us to test assumptions about data flow, user interaction, and notification logic early in the process. Together, these components formed the first functional prototype, capable of supporting the core needs listed in the case material. This could also prove to support service processes within the broader service industry.

3.1.1 Unfolding the problem domain

To gain an early understanding of the fundamental structure of a booking system and the related data relations, we initially developed a quick prototype in which the frontend was developed using Streamlit. The purpose of this first prototype was not to create a finished solution, but rather to explore how customer data, appointments, and notifications interact in practice. This rapid prototyping approach made it possible for us to test core ideas early and reflect on the system's potential as we continuously updated the scope and end goal of the project.

Streamlit is a Python-based framework for quickly building interactive web applications, especially suited for data-driven prototypes and rapid experimentation (Streamlit 2023). Streamlit proved to be highly practical and fast-paced for developing the first prototype, as it allowed us to build an interactive web interface using only Python without needing to structure HTML, CSS, or JavaScript aspects of the code. This enabled rapid experimentation with CRUD functionality, data display, and interaction flows, which helped us understand the requirements of a booking system early in the process. Our experience with Streamlit has been positive, as the framework is intuitive, has fast-paced phases between iterations, and well-suited for demon-

strating system concepts to non-technical stakeholders. This early focus on building and testing a working prototype also reflects the Agile value of prioritising “working software over comprehensive documentation” (Beck et al. 2001a). However, we also observed its limitations in terms of customisation and scalability.

Based on the insights gained from unfolding the problem domain and working with the first prototype, we used the FACTOR framework to define what the system was meant to solve in its initial form. This made it possible to describe the system’s purpose, scope, and overall responsibility in a structured way (Mathiassen 2018, pp. 23–25, 40).

FACTOR Criterion

In the following section, we aim to define the mission of what our first prototype is meant to solve. To contextualise the development to the case, we made the following FACTOR criterion based primarily on Figure 2.1, as well as the case study of Section 2.2 (Mathiassen 2018, p. 40).

- **Functionality:** The product should send notifications automatically, for each customer that has a scheduled cleaning appointment the next day. The system must allow for Unit Cleaning administrative staff to utilise a web-interface to book, update and delete appointments.
- **Application Domain:** This system is intended to support Unit Cleaning in avoiding missed appointments, as well as supporting the administrating employees in easily creating appointments, or changing existing ones. The system focuses on increasing purposeful and clear communication.
- **Conditions:** The system assumes that users have access to a stable internet connection, basic computer skills (website navigation, read/respond to emails, etc.), and compliance with data protection regulations such as secure handling of customer contact and appointment data. Daily operations depend on up-to-date appointment information and timely delivery of reminders.
- **Technology:** A web-based booking interface implemented in Python, HTML and JavaScript (first as a Streamlit prototype). Furthermore, the system consists of a database for storing customer and appointment data, and an email-based notification service using *Simple Mail Transfer Protocol* (SMTP) to deliver automated reminders to customers.
- **Objects:** Customers, employees, appointments, cleaning tasks, notification messages, and user accounts. Also, the digital artefacts that represent these objects in the database and UI (customer records, notification messages, etc.)
- **Responsibility:** The system is responsible for reliably storing and updating bookings, ensuring that reminders are generated and sent at the correct time. This is in order to reduce manual coordination effort for administrative staff. Unit Cleaning remains responsible for maintaining the data, configuring notification rules, and acting on information provided by the system.

These criteria have been an important aspect in identifying key figures, functions and conditions in which our system should, could and would be used. This has helped identifying possible paths for further development, especially in regards to the actual implementation of our first prototype.

3.2 Implementation of the first prototype

To clarify how the first prototype was constructed, the following sections describe its main technical components of the database, frontend and backend, and their individual roles in the system.

Building a Database

To gain a base understanding of the necessary information we need to store, we created a MySQL database. Our MySQL database serves as the storage layer of our first prototype, ensuring consistent and structured handling of customers and their cleaning appointments. It stores information such as customer details (name, address, email) and appointment records (customer ID, location, date, time). This data is needed for record bookings and send notifications, which we have visualised as "Customer Information" in Figure 2.1. A relational schema, which can be seen in Figure 3.1, is used to visualise clear links between customers and their appointments, enabling reliable queries, updates, and integration with the frontend and the notification module. This information on customers and their appointments are navigated and changed with two categories of queries: Read and Write. Any Read query simply displays already existing data within the database, while Write consists of create, update and deletion queries.

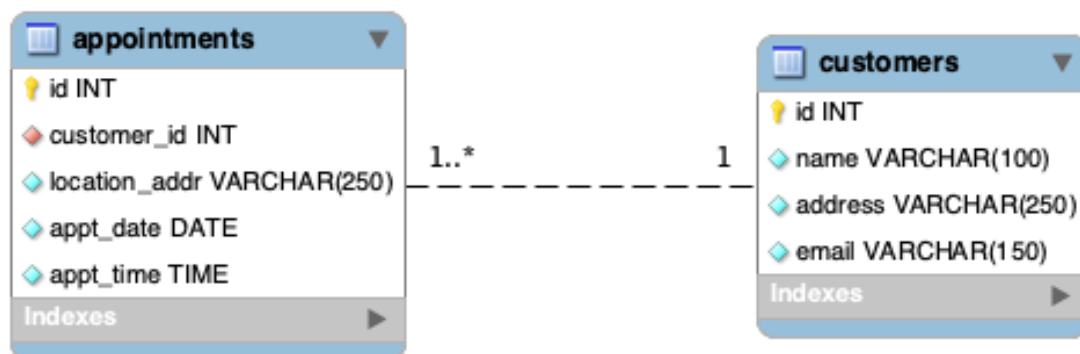


Figure 3.1: ER-diagram for database, prototype 1

The Streamlit Frontend

The prototypes frontend has been created with the intention of manual creating of customers, and creating a cleaning appointments, working with the Update Info and Read Info aspect in Figure 2.1. This means we created fields in the Create Customer, including name, email, address, and their cleaning date. The cleaning date could be updated in "Update cleaning date" as can be seen in Figure 3.2. Following the case description (see Section 1), the system has been made to fit the role of an administrative employee, giving access to create customers, delete the customers from the system and see all current customers in the database. It should be noted that the database was not connected to the frontend, and that at this stage, the focus was more on gaining insight to the necessary features and identifying the necessary input fields that were needed for the creation, updating, and deletion of the customers in the database.

The actual connection between the different databases and the frontend became redundant at this time, as the focus was more on the UI elements and what needed to be accessible on the frontend.

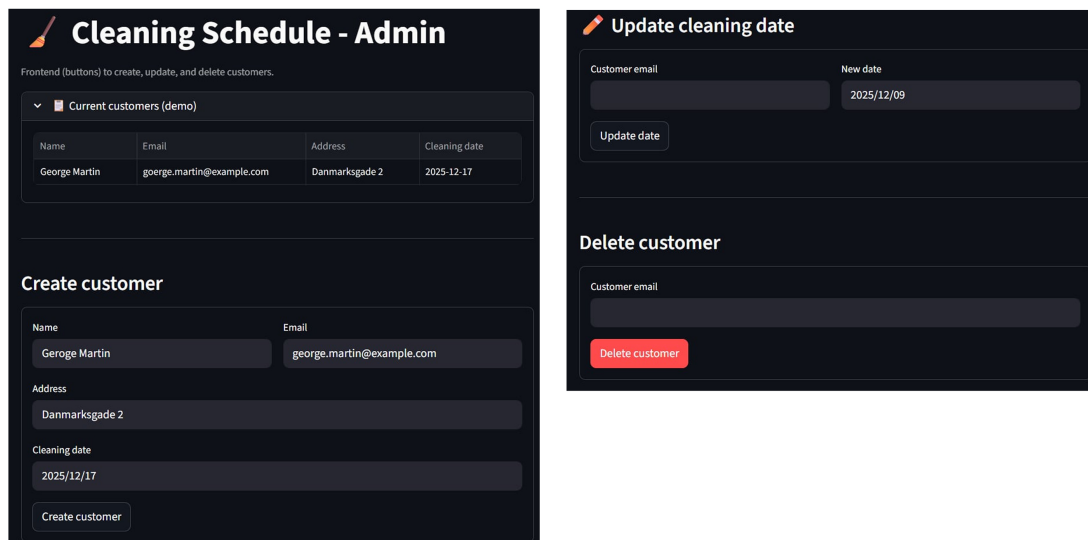


Figure 3.2: Screenshot of the frontend of prototype 1

The Backend

The backend consists of a few main elements that the project uses as a base. As mentioned, the different aspects are not fully connected, which means that the different functions are separated, allowing us to focus on the basic principles of each section. The first section is about the email, mentioned in Figure 2.1 as "SMS inconsistency". Email was chosen as the notification platform because it provides a simple and easily implemented communication method suitable for rapid prototyping. While SMS or phone-based reminders may be more practical in a full production system, email allows us to demonstrate the core idea (automated appointment reminders) without requiring additional infrastructure or smartphone devices and the expenses that come with sending test reminders via SMS. Our case partners agreed that this made more sense for testing purposes. To create the email in the prototype, we use Python's built-in `smtpplib` library to construct and dispatch messages via an SMTP server. Notification messages are generated dynamically based on upcoming appointments retrieved from the database, and the system includes a dry-run mode that prints the messages to the console instead of sending them. This allows safe testing without affecting real customers and easy debugging. To avoid exposing sensitive information (including SMTP credentials, database login details, and email passwords) the prototype stores all confidential configuration values in a separate file we named `private.settings.py`, which is excluded from the general control of the app. This prevents accidental leakage of secrets through Git repositories or shared project folders. Even in an early prototype, these precautions are essential, as handling user data and automated notifications requires responsible security practices even at the very beginning of the development process.

Lastly, the business logic of the Streamlit frontend is created to deal with validation errors, such as having the correct date format and requirements for emails. It also simulates the basic aspects of the delete and update functions, allowing basic interaction with the frontend and the memory. The frontend uses an in-memory Array, saving the data temporarily while the web application is running. This allowed us to work on the database, email and frontend separately, enabling us to get an overview of the different elements and what each section needed for further implementation.

3.3 Insights From the First Rendition

The intention behind the first prototype was to digitise the most basic parts of the booking process. This included the registration of customers, the storage of appointment information, and the manual handling of reminders. The primary goal at this initial stage was to support Unit Cleaning in reducing manual administrative work and creating more efficient internal workflows; consequently, the focus was squarely on digitalisation rather than automation. Working with this initial focus gradually shifted our perspective. It became clear that the real potential for value creation did not lie solely in digitising existing work processes but rather in further automating them. This realisation marked a key turning point in the unfolding of the problem. Instead of focusing only on how booking procedures could be supported digitally, we began to ask how they could be automated in ways that further reduce manual work and add more intelligence to the system. According to Sommerville, automation plays a central role in modern software systems by reducing human error and increasing efficiency in repetitive tasks (Sommerville 2016, pp. 23–25).

After completing the first prototype, we presented and discussed its implementation with our case partner and demonstrated how the initial requirements and functionalities had been addressed. Based on feedback from the company, as well as discussions with our supervisor and within the project group, we reflected on the potential of the solution beyond its original scope. These discussions led to a shared decision to further develop and expand the case to explore how additional value could be created for Unit Cleaning through the prototype. The main aspect we identified for improvement was the area of "Change appointments" (see Figure 2.1). In the first prototype, this area had not received significant improvement, as it still requires constant conversation and interaction between the customer and administrators. This results in time-consuming workflows and creates a risk of inconsistency in scheduling and reminders. Although the first prototype supported the digitalisation of these processes, it also made clear that digitalisation alone was not sufficient to significantly optimise the workflow. This fundamental shift in focus from digital support to general automation also led to a necessary reconsideration of our problem formulation.

Together, the insights from the first prototype revealed both the strengths and limitations of our initial approach to the problem. Although digitalisation supported Unit Cleaning in structuring administrative tasks, it did not change the effort required of the customer to manage bookings and communication. The continued reliance on manual coordination, especially in relation to changing appointments, highlighted a clear opportunity for further value creation. We therefore chose to rethink the role of our system. This reflection formed the basis for the second iteration of the prototype, where the focus shifted towards using AI to automate interaction and reduce the dependency on manual administration.

4 Second Prototype

This chapter presents the development of the second prototype, which builds on the insights gained from the first iteration of the product. After identifying the limitations of a simple digitalisation, we shifted our focus towards automation and began exploring how an AI-supported interaction model could support, and in some cases replace, the manual administration previously required between customers and administrators. The following sections explain how we defined the problem domain for this new direction, how user stories shaped the development, and why AAI emerged as a design contribution to the project and overall system design.

From this point onward, references to the project or prototype refer to the second iteration unless stated otherwise.

4.1 Defining the Problem Domain

As we worked through the first prototype, it became increasingly clear that the core problem was not just the missing digital interface and database. A significant amount of manual work was still required to coordinate appointments, update schedules, and keep reminders consistent. Following agile development principles, we tried to stay open to changing our understanding as we learned more. Agile development methods encourage teams to adjust their direction when new insights appear, instead of committing to fixed decisions too early (Sommerville 2016, p. 75). With this mindset, the feedback from our case partner, our supervisor, and our own reflections pushed us to reconsider what features the system actually needed. *Agile Project Management* (APM) also highlights the importance of continuous collaboration with stakeholders, so priorities can evolve and be specified during the project (Sommerville 2016, p. 84). Our conversations with the company pointed in the same direction. Increased automation, and not just digitalisation of their processes, could create more value, by significantly reducing the manual coordination that caused the problems in the first place.

When we revisited the rich picture (see Figure 2.1) from Chapter 2.2, the “change appointment” process stood out as the main point of friction in the workflow. Even though the prototype digitalised parts of the process, it still relied heavily on communication between customers and employees, which made the workflow slow and inconsistent. Sommerville 2016 notes that user stories and early prototypes often reveal gaps or new opportunities once the system is seen in action (Sommerville 2016, p. 77). The first prototype helped us understand the domain in a way that was not obvious from the initial case description and gave us a clearer picture of where the inefficiencies were.

At this point, we then needed to define more precisely what the system should handle and what belonged outside its responsibility. To do that, we created a *system context*, depicted in Figure 4.1. In the book *Object-Oriented Analysis and Design*, a system context outlines the environment the system interacts with and clarifies where its boundaries are (Mathiassen 2018, p.23). It maps out the surrounding actors and dependencies that shape how the system behaves. Establishing this early is important because it shows which behaviours belong inside the system and which should remain external (Mathiassen 2018, p.40).

To make the domain easier to analyse, we also created a *mini-world*. In database and systems design, a mini-world is a smaller, controlled representation of reality that includes only the concepts relevant to the system being developed (Elmasri and Navathe 2017, p.4). This helped us isolate the “change appointment” scenario and study it without potentially unnecessary data in the organisation. It follows the idea that conceptual modelling helps designers understand the essential interactions of a domain before deciding how a system should support them (Elmasri and Navathe 2017, p.38). Working with the mini-world made it easier to see where problems in the workflow were and how the system might eventually handle some of the tasks that previously required an administrator. This shift in perspective gave us a clearer direction for the second iteration. Instead of designing a system that only supported employees in managing appointments, we began exploring how the system itself could take over some of these responsibilities.

Having redefined the system boundaries and clarified the core problem of the domain, the next step was to understand how actual users would interact with the system in everyday situations. We needed a way to translate the theoretical insights from the mini-world, the system context, and the rich picture into concrete interaction requirements. This is where user stories became essential.

4.1.1 User story deliveries

User stories were chosen as a deliberate design method to bridge our domain understanding with the system behaviours we needed to define. In agile and human-centred development, user stories help connect user needs with technical decisions by expressing goals from the user’s point of view instead of through rigid specification formats (Cohn 2004, pp. 3–16). This approach allowed us to move from abstract ideas about automation to more concrete expectations about what the system should do. We did not develop a full story map, but our approach aligned with Patton’s idea that user activities should be understood as connected narratives rather than isolated functions, which can be seen in Appendix E (Patton 2014). Thinking of the stories as small narratives helped us focus on why users would take certain actions and how the system should respond to them. To support this, we worked with personas as a way of grounding the user stories in realistic user situations. Personas represent typical users within the problem domain and help reason different goals, behaviours, and constraints without relying on real users directly (Interaction Design Foundation 2025). The personas used in this project are available in detail in Appendix E. By writing user stories using the named personas, we were able to maintain a consistent user perspective across the design process and avoid framing the requirements purely from a system standpoint. The stories also helped reveal differences between users. Some interactions required stable preferences that the system should remember, while others involved dynamic factors such as weather, calendar availability, or shared access across multiple homes.

Persona 1: Jonas

Jonas represented a user with an irregular weekly routine and minimal interest in receiving notifications. His availability depended on work, guests, and weather conditions. To support this, the system checked his personal calendar before suggesting any appointment and only proposed times that did not overlap with existing plans. This emphasised the need for adaptive scheduling that responded to the user's context. Notifications also played a central part in his interaction. Although the system supported reminder emails, Jonas chose to disable them entirely. He preferred fewer messages and allowed cleaners to enter without his presence.

As Jonas, I could ask the AI-agent to remember that I require pet-safe cleaning products, so I would not have to repeat it each time I booked an appointment.

As Jonas, I could receive cleaning recommendations based on my plans and the weather, so my house would stay clean and ready for guests, changes in schedule, or anything related to my dog.

Persona 2: Anna and Mikkel

Their case involves a more complex interaction pattern involving multiple addresses and shared booking access. They moved between two homes, and their cleaning needs changed depending on their location, the weather, and whether the summerhouse was being rented out. Their notification preferences differed significantly from Jonas'. They activated two reminder types: one email sent 24 hours before an appointment, and another sent three hours prior. This helped them remain aware of upcoming appointments and ensured they were present when cleaning was scheduled for their private home. Their case reinforced the need for flexible reminder timing and user-specific schedules that worked across multiple addresses.'

As Anna and Mikkel, we could use the website to get recommendations for cleaning dates when heavy rain was expected near our summerhouse, so we would not need to check the weather ourselves.

As Anna and Mikkel, who owned and rented out two homes, we could switch between cleaning schedules to be able to manage the specific needs of each property quickly and easily.

Takeaways

The user stories presented a clear picture of how different users interacted with the system and how these interactions shaped its requirements. Jonas' case showed the importance of personalised scheduling that relied on calendar checks, weather forecasts, and stored preferences such as his need for pet-safe cleaning products. His behaviour emphasised the value of an adaptive system that supported a notification-free workflow. Anna and Mikkel's case highlighted the need for reliable multi-address management, weather-dependent recommendations, and synchronised appointment data across two users. Their reliance on multiple reminders showed that users with more complex schedules required consistent oversight supported by flexible notification options.

As we worked through the scenarios, our understanding of user interaction developed further. We identified contextual requirements such as choosing the correct address for each appointment, handling shared access between users, and offering customisable reminder settings. This pointed towards a need for a more flexible interaction model.

4.1.2 Users and Agentic AI

Knowing that the users of our system would have such differing needs, we have sought to find ways to better the UX. Finding a flexible system that could allow for ease of access, as well as provide tailored assistance for different user types. This revelation led us down the path of AAI; an AI-agent appearing as a chatbot on the UI, onto which we can build tailored functionalities.

With the continued expansion of AI into more and more systems, we are experiencing a 'regression' of sorts within the UX design approach to GUI applications. Said expansion of AI includes primarily AI-based assistants on websites or systems such as Microsoft's Copilot, being integrated into the Office programs (Virvou 2023). Thus, many interfaces move away from the well-established and user-friendly GUI, based heavily on visual key pointers such as icons and buttons, in favour of aspects of a more text-based interface - the CUI (Jacob Nielsen 2023a). Usually, however, CUI tends to be user-hostile, posing needs for specialised knowledge of functions, as opposed to GUI where intuitive elements are key. However, as said shift happens towards an AI-based CUI, users get more exposure to a system that seemingly supports the mentality of "do as I mean, not as I say", as AI can better take natural language and extrapolate the necessary functions from said natural language. Therefore, this shift from a mainly visual-based interface to having AI text-based interfaces seeks to more accurately support the intention of users through prompts written in natural language, as opposed to traditional CUIs, which require very precise commands to function (Geeksforgeeks.com, n.d.). As the average tech-user is currently used to user-friendly GUIs, which have been the dominating UX for the past centuries, it results in a massive shift in paradigms to return to an interface that usually relies on the user's ability to correctly articulate their intentions (Geeksforgeeks.com, n.d.).

All of this is to say that the expectations of how users experience the digital world are shifting with the introduction of AI, with them being used to GUIs, a user's interaction with systems through visual aspects (Jacob Nielsen 2023a). With this change, there grows a need to rethink the competencies needed to fully utilise these products. Jakob Nielsen, holding a Ph.D. in human-computer interaction as well as being co-founder of the Nielsen Norman Group, argues that with poor literacy skills within the adult population, users lack the ability to articulate themselves well enough to effectively utilise the concept of intent-based interfaces as they currently exist (Jacob Nielsen 2023b). This sets an expectation that to best integrate and utilise AI, there must be a focus on ease of use and guiding users on how to interact with the new interface (Jacob Nielsen 2023a). Despite these challenges, users of many applications are encountering more AI tools over time. This is why we argue that it is beneficial to focus on continuing to make use of visual aspects within UIs, where interaction will be both prompt-based with CUIs, and command-based with GUIs, birthing a hybrid UI (Jacob Nielsen 2023a).

In Section 4.1, we redefined the case at hand to not only be about automating communication, but also to be about giving the customer an interface in which most, if not all, of their contact with the Unit Cleaning should occur. This interface should not simply be assisted by an AI-agent, but rather be an AI-agent assisted by visual aspects within the UI. As we aim to improve UX, we should heed the idea of a hybrid UI. This would allow for the prompt-based user interaction to be enabled in a UI, though the exact elements are to be expanded upon in Section 5.1 & 5.2. Utilising this combination of elements, we aim to make a platform with which the user can interact with an AI-agent. For clarity's sake, having widgets or visual displays supports the information provided by the AI-agent, with lesser intractability to ensure the AAI being used first and GUI components second. However, we recognise that users can be less willing or literate to utilise the AI-agent (Jacob Nielsen 2023a). We have therefore deemed it crucial to provide the opportunity to, in one way or another, avoid the use of the AI-agent and return to a regular manual online booking system.

4.1.3 Problem domain summary

Based on the redefined problem domain, we made some changes to the factor criterion from Section 3.1.1. The main difference is the change in who uses the system. In the first prototype, the user is the administrative staff, as mentioned before, and with the new changes, the user becomes the customer. This changes Functionality, Application Domain, and Responsibility. Changes in the first two categories mean changes to the interface of the application, together with what system access will be needed, as administrative staff typically have more access than a single customer. The last lies in that, rather than reducing the effort, this version eliminates the staff's workload as the staff no longer functions as a middleman. The system now has interactions between the customer and the system directly. This change can be seen in Figure 4.1. The figure illustrates the system context of the second prototype, with four key entities, the user, the system interface, the central server and the employee. In this figure, the staff becomes a secondary part of the system and the "Server" becomes the main hub of the program, which is opposite of the Figure 2.1. This is the reason the employee area is faded, as the prototype doesn't handle the aspect of providing information to the employee, who is the cleaner, as the main aspect is handling the experience of the user with the system. The Employee would need a second type of interface, where they have access to different types of information compared to the user.

All the above lead to the basic requirements of the system, used to solve the problem. As we increase the focus on automation through AI, it becomes a requirement of the system to be able to effectively handle the inputs/outputs of the system. This means ensuring the frameworks used can provide necessary control for scalability, reliability and maintainability. The system also needs to be customer-focused and have the ability ease the UX with a system. These revised criteria formed the foundation for the design and implementation decisions described in the following chapter.

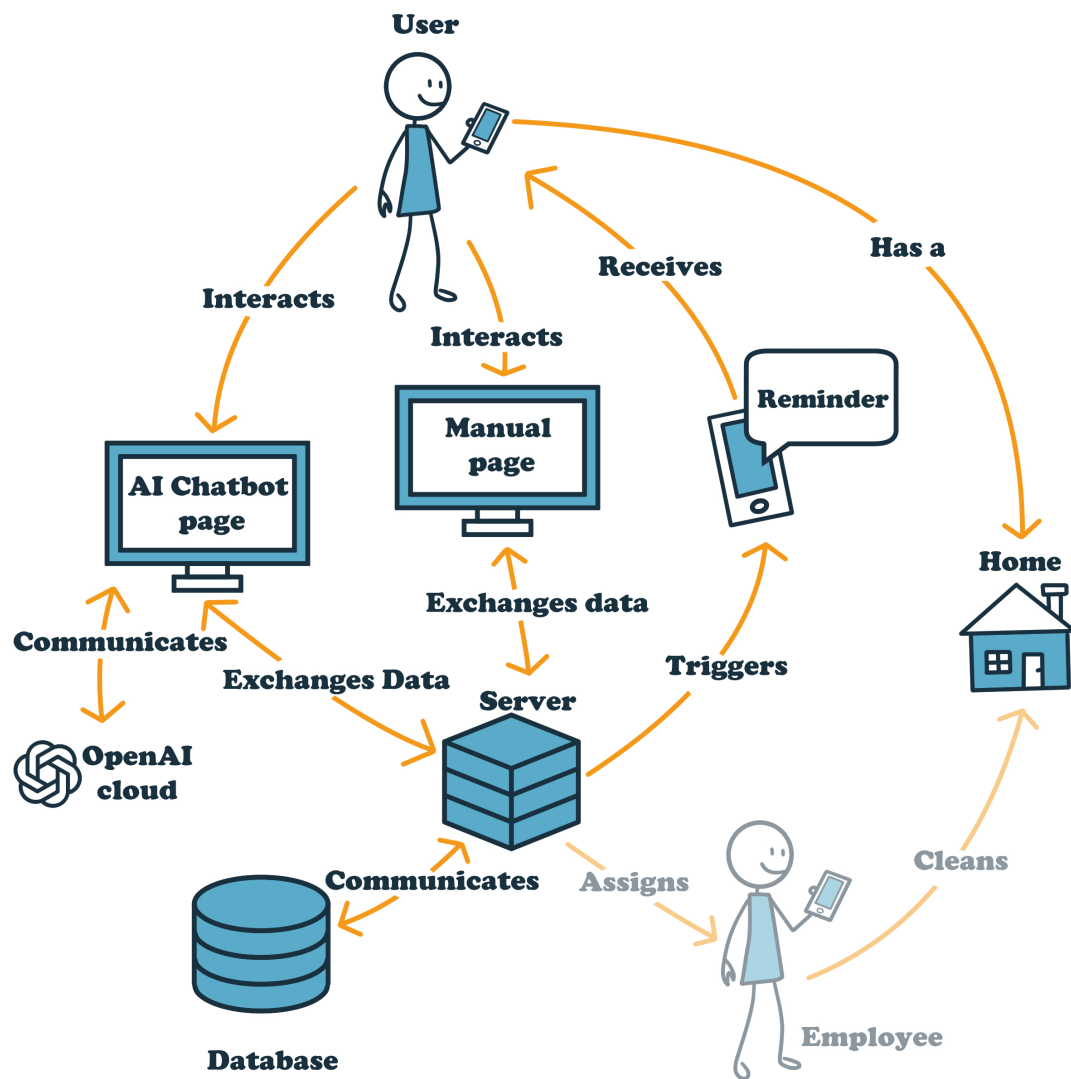


Figure 4.1: Rich picture of the system context.

5 Application domain

Building on the redefined problem domain and design direction from the previous chapter, this chapter focuses on the application domain and clarifies the context that will shape the system which we are designing for the second prototype. Here, we draw on system development principles, including inspirations from ASE, to explain how the project evolved from its initial iteration to the final solution, which we present. We will outline the design choices that guided the development, how these choices are reflected in the system architecture, and how the backend requires structure to support cohesion across components. Finally we describe the new features implemented in this process and explain the rationale behind them.

5.1 Design choices

The design choices behind the second prototype are based on insights gained from the first prototype, which primarily focused on digitalisation rather than automation. As discussed in Section 3.3, this approach reduced some of the manual work but did not address the underlying issues in the workflow. Moving towards automation, therefore, required a restructuring of the system and a reconsideration of who it should primarily support. This shift had consequences not only for functionality but also for how responsibility and control were distributed between the user and the system.

Instead of designing the system around administrative staff, the second prototype is centred on the customer as the main user. This shift influenced how the problem was modelled to fit the system, particularly through revisiting the objects and responsibilities defined in the FACTOR criteria in Section 3.1.1. By revisiting the revised problem domain in Chapter 4, it became clearer which actors interacted with the system, which events triggered changes, and which responsibilities belonged inside the system context rather than outside it. The updated rich picture, see Figure 4.1, reflects this change by positioning the server and AI-agent as central components, while employees are moved to a peripheral role.

In terms of functionality, these considerations pointed towards an automation process capable of interpreting user input, coordinating bookings, and handling changes without manual intervention. As we described in Section 4.1.2, the system needed to support natural language interaction while remaining functional as an AI-agent, able to perform specific tasks. The design choices constitute the basis for the aspects worked in detail later in the chapter, ensuring the focus on the necessary solutions for each aspect, described in Figure 4.1.

To build an understanding of what the next step in the design process should address, we sought to define the system's boundaries and the content of the objects. This allowed for the utilisation of use cases in the following section, which translate these early design choices into practical scenarios that guided the development of the prototype. To make these choices more concrete, we translated them into use cases describing how users would interact with the system in practice.

5.1.1 Use case

To move from abstract design principles to concrete system behaviour, we utilised use cases to describe how users could interact with the system in practice. To determine which functionalities should be included in the updated prototype, we worked from our user stories from Section 4.1.1. The user stories helped us identify potential users and their needs; these aspects were then used in the development of our use cases. Use cases provide a structured overview of who our actors are and how they interact with the system (Mathiassen 2018, pp. 121-122). We developed the use cases to explore how different types of users could engage with the system, and how their individual needs shaped the booking logic, the behaviour of the AI-agent, and the requirements for reminder notifications. Building on the user stories, the use cases helped us identify where the workflow needed refinement. This made it possible to outline the components needed for correct appointment scheduling and personalised recommendations.

To highlight possible contrasting needs, we selected two different types of users. 'Jonas' represented a user with an irregular weekly schedule, minimal interest in notifications, and a preference for low-interference interaction. Besides this, 'Anna and Mikkel', illustrated a scenario involving two homes, shared access to bookings, and a stronger reliance on system-generated reminders. Comparing these cases broadened our understanding of how the system should operate across varied real-life contexts rather than relying on a single idealised user pattern.

While the Anna and Mikkel case plays an equal part in the implementation of features, as the use cases collectively are made to cover the central scenarios that our system must support, we will go in depth with the user story of Jonas for the report. This is because Jonas's scenario is more straightforward to explain and provides a clear, representative example of our analytical method, making it more efficient to detail the process without the redundancy of explaining two cases that were developed using the same methods.

To convey the overview of the usage details, we make use of a Use Case Specification (Figure 5.1), and a *Use Case Diagram* (UCD) (Figure 5.2), in order to provide an overview of the process (Mathiassen 2018, p. 129). Figure 5.1 is a description of the user type of Jonas, where booking an appointment is shown with the intention of providing details of the user's needs.

Use Case: Jonas booking appointment

Goal: Jonas requests a cleaning appointment via the AI-agent. The system checks constraints and proposes an appointment that requires minimal input for confirmation.

Information Gathered/Read:

- Time and Date
- Service Type (e.g., Pet-safe cleaning)
- User Preferences (minimal notifications, irregular schedule)
- External Calendar Availability
- External Weather Data (DMI)

Objects: Customer, Appointment, Address, Calendar, WeatherData, AI-agent.

Functions: request booking, check calendar conflict, query weather data, propose appointment, confirm booking.

Figure 5.1: Use case specification for "Jonas booking appointment"

The use case of Jonas is also represented in a UCD (see Figure 5.2). This diagram organises the system into two functional subsystems: Preference management and booking service and further exemplify the interaction with the system. As illustrated, Jonas acts as a primary <<actor>>, who initiates the workflow. To achieve the goal of an optimised booking, the system coordinates with several <<secondary actors>>: the AI-agent for processing logic, DMI for weather data and the calendar service for availability. The relationship between "request recommendations" and "Confirm Appointment" is defined as <<include>>, signifying that the confirmations is mandatory step in the recommendation process.

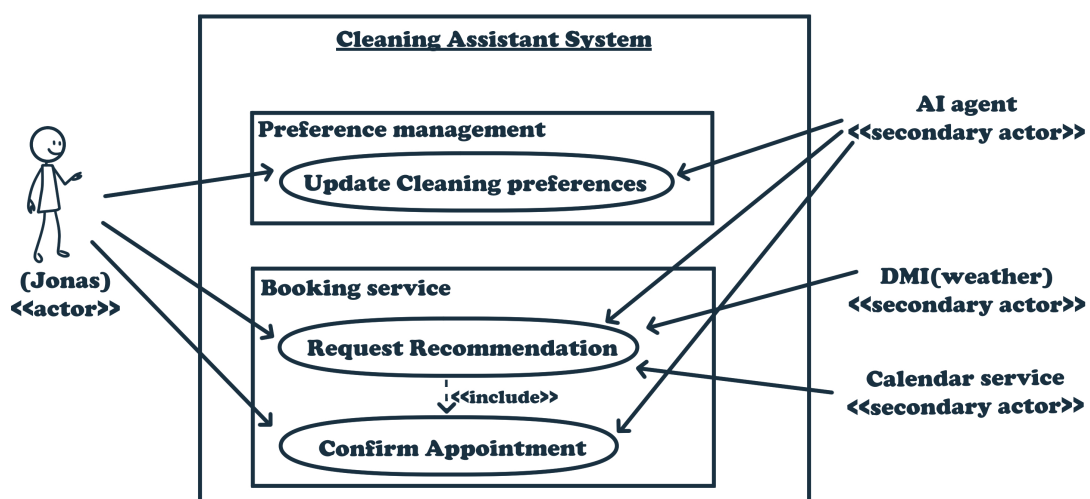


Figure 5.2: Use case diagram of Cleaning Assistant System"

The combination of the use case specification and the UCD provides a comprehensive view of the system's functional scope. The structural overview ensures the AI-agent's decision-making process is directly aligned with the user's needs. Ultimately, these use cases formed the basis for identifying the requirements that guided the development of the second prototype.

5.1.2 Requirements

The requirements for this project were not established as a fixed upfront list, but were developed iteratively throughout the process, reflecting the philosophy of agile development, stated as *"Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage."* (Beck et al. 2001b). This iterative process allowed the development phase to focus on the immediate implementation and necessary exploration of features, ensuring that requirements always aligned with the evolving understanding of the problem domain and the focus on the customer.

Based on the changes outlined in Section 4.1, specifically the elimination of the administrative middleman and increased focus on automation through AI, the requirements fall into two main categories:

1. Functional requirements

Functional requirements are driven by the need to fully automate the booking process and provide a direct customer experience.

- AI-Supported booking: the system must allow the user to book, modify and cancel services using natural language
- Manual booking fallback: the system must provide a traditional, form-based interface to ensure the user has full control and provide an alternative method for service booking.
- Booking logic: the system must process the interface accurately and generate a booking while placing it in the database.
- customer feedback: the system must automatically send a reminder to the user, based on their preferred time.
- preferences: the system must allow the user to set personal preferences for service types, frequency and communication type.

2. Non-functional requirements

These requirements ensure the system directly delivers on the goals of Operational Efficiency, Service Reliability, and Business Scalability outlined in Section 2.1.1.

- Scaleability: The architecture must support scaling of the server component to accommodate future business growth and increased bookings without requiring an increase in administrative personnel.
- Reliability: The core booking process must be highly reliable, ensuring that no confirmed booking is ever lost, mislogged, or unassigned due to system errors, while also ensuring no customer misses the appointment due to reminder errors.
- Efficiency: The system must provide an easily accessible, anytime booking service. Enabling users to book at their convenience.

- Usability: The customer interface (AI-agent and Manual Page) must be intuitive and easy to use, making the booking experience smoother than the previous staff-mediated process.
- Maintainability: The underlying code architecture must be organised to allow future developers to easily add new features without breaking existing functionality.

These requirements create the basis for the implementation, where they are further specified through the iterative process of each of the features. These examples will be described in the next sections on implementation.

5.2 Implementation of the second prototype

We used the identified requirements to ensure a common understanding of what the product must be able to cover and be responsible for, as an organising tool in the development process. The remaining chapters detail the implementation of the second prototype, focusing on the system's architecture, core components, and the specific functional and non-functional features of the system.

5.2.1 Architecture of system

With the knowledge of the user stories, use cases, and the requirements, we created an architecture for the system shown in Figure 5.3. The architecture is designed using principles from *Object-Oriented Analysis and Design* (OOAD), in particular, the chapter on components (Mathiassen 2018, p. 191-209). The architecture establishes a clear separation of concerns through three conceptual levels: "interface", "backend", and "external". These layers each describe different aspects that, as a whole, illustrate the main components of the system. At the centre of the architecture lies «Component» Backend, which represents the system's core. This layer is the function layer, which contains all the backend logic handling the booking of appointments. Within this the «Component» AI-agent is placed as a central component, as it has a substantial role in coordinating booking, interpreting user intent, and providing recommendations.

To support these responsibilities, the booking application interacts with a set of «Component» External components that provide contextual information and persistent data. These include components for weather data (DMI), calendar, database, and OpenAI's language model services. The primary responsibility is to encapsulate and manage all interactions with services and resources existing outside the intimate system boundary. These external components are not part of the interface layer but serve as supporting services that the AI-agent and booking logic draw upon when evaluating booking requests or changes. For example, weather data and calendar information enable the system to adapt bookings dynamically, while the database ensures that customer information and appointments are consistently stored and retrieved. The OpenAI component provides language-processing capabilities used by the AI-agent to interpret user input, but it does not itself contain system logic or decision-making responsibility.

The upper level of the architecture consists of the «Component» Interface components, represented by the manual page and the AI-agent page. These interfaces provide two alternative interaction modes for the user. Overall, the architecture reflects the redefined system boundaries and the increased focus on automation identified in the problem domain. By concentrating coordination and decision-making within the booking application and AI-agent, the system is structured to replace the former administrative middle layer while supporting flexible, customer-driven interaction. The specific design choices and implementation details of each component are elaborated further in the following sections.

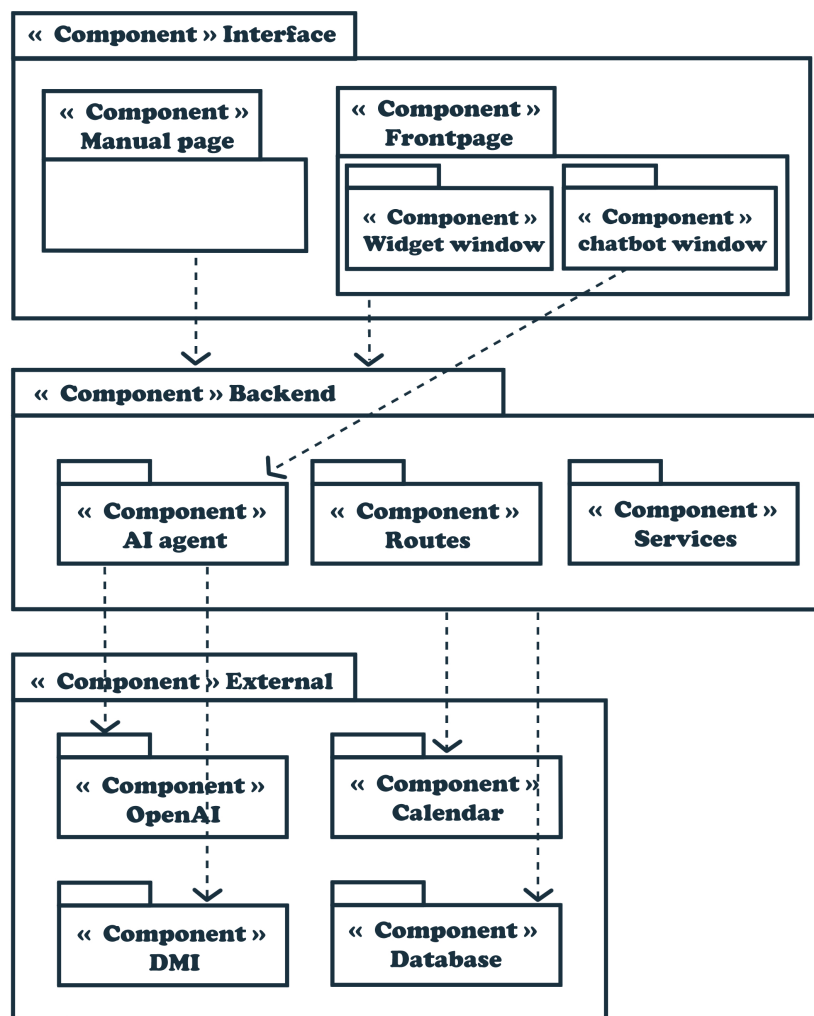


Figure 5.3: Architecture of the second prototype

5.2.2 Backend

The systems core operation is handled by the backend. This consists of a modular structure, ensuring the separation of concerns. The backend code is organised in two primary sub-directories: `routes` and `service`. This organisation ensures clear responsibilities: `routes` handles the incoming requests and define API endpoints, while `services` contain the business logic and data processing, working with the database. This separation makes the codebase maintainable, testable and scalable as the system grows.

Notification system Our notification functionalities are designed to inform customers about their upcoming appointments. This is one of the functional requirements, as covered in Section 5.1.2, derived from the initial conversations with Unit Cleaning for the first prototype. The design enforces strict separation of concerns by delegating all data retrieval and rule enforcement to the Business Logic Layer. This means the notification code doesn't directly access the database; instead, it retrieves the necessary appointment information by calling the necessary functions made for database interaction. This design ensures that the notification module focuses solely on the delivery mechanism, while the business logic defines the rules for filtering and formatting the data (e.g., notifying customers).

MySQL / Database

The database was designed to support a flexible and structured data model that could adapt to the needs identified in the use cases. This was achieved by containing customers, addresses, their preferences, appointments, and services as individual entities, ensuring that the database can support shared households and changing customer needs.

Figure 5.4 is an ER-diagram that shows the structure around a set of core entities and their relationships. *Customers* and *addresses* have a many-to-many relationship, connected through the junction table *lives in*, allowing multiple customers to be associated with the same address and vice versa. *Addresses* are related to *appointments* in a one-to-many relationship, as each address can have multiple appointments, while an appointment only belongs to one address. Similarly, *addresses* are connected to *preferences* so that an address can have as many preferences specified as needed. Services and appointments are linked through the junction table *has ordered*, representing a many-to-many relationship. Together, these relationships define the cardinality and participation between the entities in the database schema.

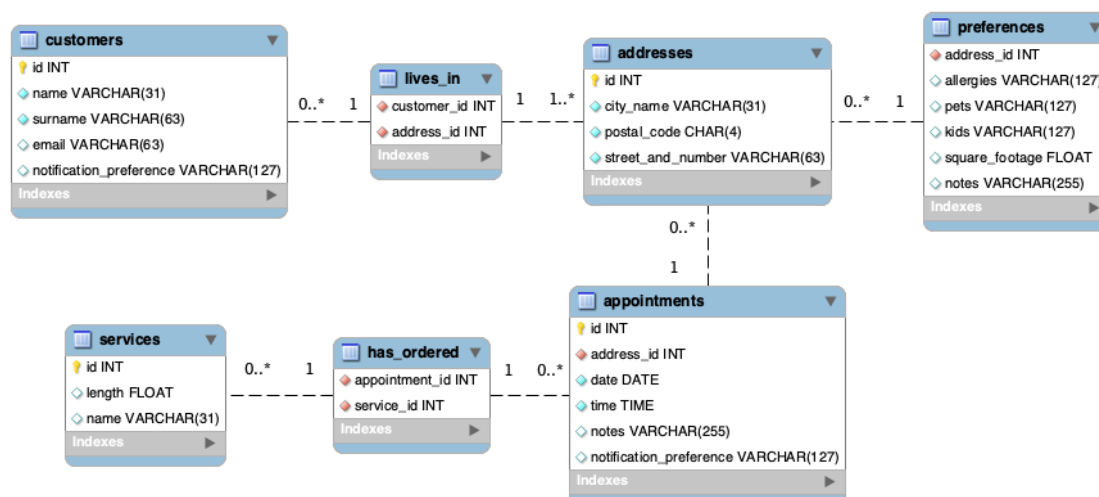


Figure 5.4: ER-diagram of the database for second prototype

We chose a many-to-many relationship between customers and their addresses to more closely model the mini-world we defined (See 4.1), where a person can have multiple homes, and multiple people can share a home, like in the Anna & Mikkel persona. Information like notification preferences is unique from customer to customer and therefore belongs to their table, while preferences about cleaning more accurately depends on the address, with how the Jonas persona wanted pet-safe cleaning products specifically at their address with a pet. The database's table contents and relations are designed to model realistic relations.

Calendar

One of the primary aspects in both visual guidance, as well as functionality in the context of providing suggestions of cleaning dates, is the calendar. The calendar widget is structured around a clear separation between internal appointments stored within our database and events originating from the user's personal Google Calendar. This division makes it possible to manage system-specific scheduling logic independently of external calendar data, while still presenting both sources as a single, coherent calendar view to the user. As described earlier in Figure 5.4, appointments in the system are not directly associated with individual customers, but are instead connected to their addresses. This design choice allows users to access all appointments relevant to their person across one or more addresses, without requiring duplication of appointment data or customer-specific mappings. Internal appointments are retrieved through a dedicated backend endpoint, which consists of the database logic needed to assemble a complete overview of upcoming appointments.

Beyond this, the system ensures that the retrieval of internal appointments from the database runs in parallel with the integration of the user's personal Google Calendar through Google's *Application Programming Interface* (API). Access to Google Calendar data is established using an OAuth 2.0 authorisation flow, where the user explicitly grants consent via Google's consent screen, as seen in Figure 5.5 (Google, n.d.).

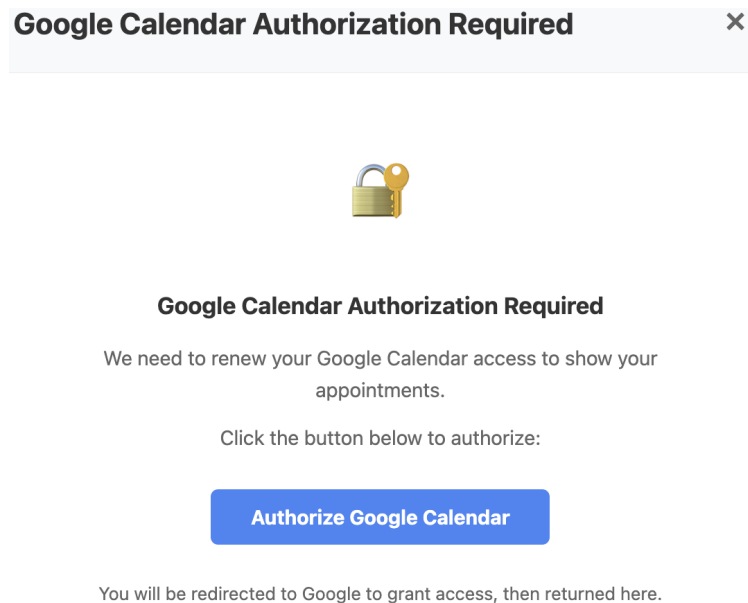


Figure 5.5: Calendar Widget call for authorisation

The resulting access and refresh tokens are stored server-side, enabling continued access to calendar data across sessions without the need for continuous user authorisations. All communication with the Google Calendar API is encapsulated within a dedicated service and can be maintained or replaced without affecting other components. When retrieving events from Google Calendar, the system handles token refreshing automatically and provides structured fallback behaviour in cases where authorisation has expired or become invalid. Rather than failing silently, the backend returns a specific response that allows the frontend to prompt the user to re-authorise through the same OAuth 2.0 authorisation. Events retrieved from Google Calendar are normalised into a simplified internal representation, aligning them structurally with the appointments originating from the system's own database, which then results in an interface that looks as shown in Figure 5.6.

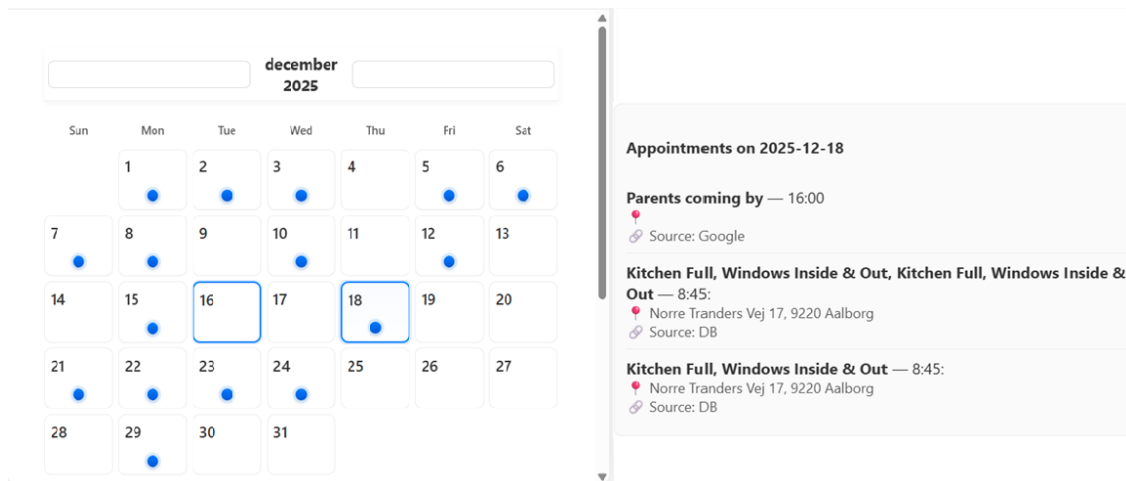


Figure 5.6: Calendar Widget of a given date

Once both database appointments and Google calendar events have been collected, they are merged server-side into a single, combined dataset. This ensures that the frontend only needs to process one unified calendar feed, assisting in maintaining consistent behaviour across clients. Calendar data is retrieved within a defined time window, currently defaulting to the upcoming weeks, which limits data volume and improves both performance and readability within the widget. The calendar widget implemented on the frontpage, therefore, functions purely as a supporting UI element to the user’s interaction with the AI-agent.

Weather

The weather module was implemented to support the system’s functional requirements of AI-supported booking and modification, as well as the non-functional requirements related to usability in the requirements in section 5.1.2. Rather than acting as an isolated feature, the module provides contextual weather data to the AI-agent, enabling more situationally aware interaction with the user.

In relation to the functional requirements, the weather module supports the AI-supported booking process by supplying real-time weather forecast information that can be used when suggesting booking times or service orders. This allows the AI-agent to propose more relevant suggestions to the user, such as scheduling a cleaning appointment for after unfavourable weather conditions are expected, while the core booking logic remains responsible for processing and storing the booking in the database.

From a non-functional perspective, the weather module primarily contributes to usability by improving the quality and relevance of AI-generated suggestions. By incorporating external contextual data, the system provides a smoother and more intuitive customer experience, which would reduce the cognitive load that will be elaborated in Section 6.3.1 by the user. See weather from frontend: Figure



Figure 5.7: Weather forecast frontend

From the technical perspective, the weather module runs as part of the system backend and is implemented within the services module. It uses weather data provided by the Danish Meteorological Institute (DMI), which offers publicly available weather APIs. The weather data that the module retrieves is made available to the AI-agent as contextual input for generating suggestions and recommendations.

5.2.3 AI

The use of an AI-agent was, as previously mentioned, a step in order to further the goal of automating the booking practice, through the lens of the customer’s interaction, rather than an administrative employee. To do this, we researched and worked with different entry-points and design concepts of what an AI-agent would act like within Unit Cleaning’s case (see Section 4.1.2).

Our initial approach was to connect our prototype to Cursor, which functions both as an AI-assisted *Integrated Development Environment* (IDE) and as a *Model Context Protocol* (MCP) host. Through an MCP, Cursor enables external systems to expose their own tools to AI models running on a Cursor-managed server. This setup became the first version of the communication layer between the prototype’s AI-agent page and OpenAI’s cloud services, as shown in Figure 4.1. In this configuration, Cursor handled much of the orchestration: it hosted the tools, forwarded requests to the chosen *Large Language Model* (LLM) (ChatGPT-4o), and returned the model’s responses. For quickly getting a working AI-agent integrated with our prototype, this was effective and required relatively little infrastructure work on our side.

As we started defining more specific roles and behaviours for the AI-agent, the limitations of this approach became clearer. The more we wanted the AI-agent to follow domain-specific rules, query our database in structured ways, and respect particular decision criteria, the more important it became to have precise control over the tools and their schemas (Cursor, n.d.). This led us to the tools-based paradigm, where the AI-agent’s capabilities are expressed as explicit, typed functions that the model can call. Tools in this sense can either be predefined by the model provider (such as OpenAI’s web- or file-search tools) or fully custom JavaScript object notation (JSON) schemas defined by the developer (OpenAI Platform, n.d.).

We chose to work with custom tools so that we could tailor ChatGPT-5 specifically to Unit Cleaning’s needs. Instead of relying on general-purpose web tools, we developed system-specific operations such as reading from and writing to our database. The groundwork for this database integration had already been established in the first prototype, as can be seen in Section 3.1.1, which made it natural to promote these existing functions into AI tools. An overview of the structure of the use of AI can be seen in Figure 5.8 . This figure shows the use of the OpenAI cloud being used together with our custom tool in the Flask box, which are then displayed on the frontend.

Once our tool definitions and their implementations lived inside our own backend, Cursor no longer provided unique functionality beyond acting as an editor with AI-assistance. The tool-calling logic and the mapping between model requests and database functions were now under our direct control and independent of any particular IDE. Keeping Cursor in the loop would have meant retaining an additional layer: frontend → backend → Cursor MCP → OpenAI, even though the backend was already capable of calling OpenAI’s API and handling the tool responses itself. For that reason, we decided not to base the final architecture on Cursor. Instead, we integrated OpenAI’s tool-calling interface directly into our own system and disregarded the choice of IDE, as it no longer had a lasting impact on the functionality of our system. This shift allowed us to reduce complexity, avoid unnecessary dependencies on a specific development environment, and ensure that the behaviour of the AI-agent in development closely matches its behaviour in a potential production setting.

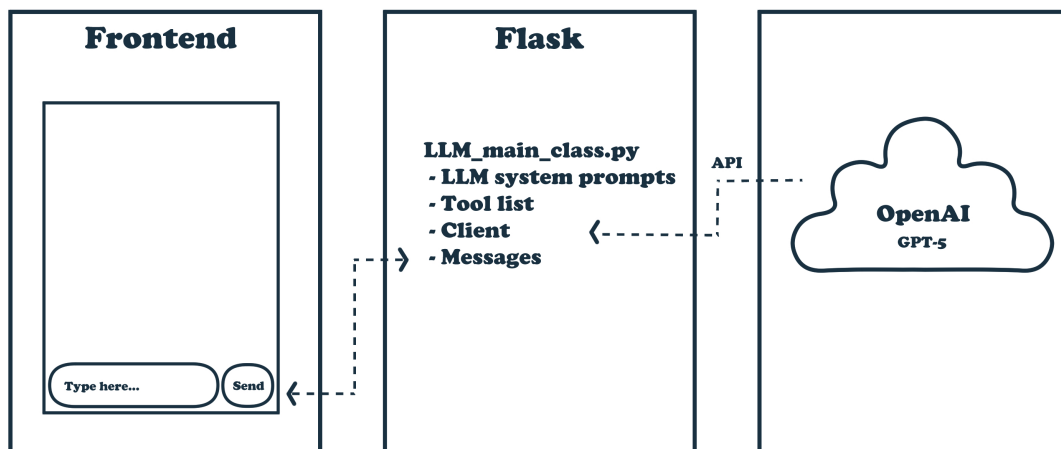


Figure 5.8: Diagram of the use of AI in the system

The structure of our AI-agent

The AI-agent we have built and utilised for Unit Cleaning follows this flow:

1. Call the LLM with the user's input, along with the AI-agent's instructions, as well as the current conversation history.
2. Detect and execute any relevant tool calls.
3. Parse the result to the conversation history.
4. Generate a text response to the user.
5. Await further user input.

As a user initiates a conversation with the AI-agent, the input is sent to the LLM to 'understand' the natural language of the input, through which any previous knowledge embedded within the AI-agent is called forth. This includes any rules we have embedded into the AI-agent itself. These rules define how the AI-agent should act, what it can and can not do, its language et cetera. In our case, the rule-set includes its role as an assistant for a cleaning firm, as well as clear instructions on how to act on what it interprets as missing information. Furthermore, it also defines the language in which the output of the AI-agent should be, making sure that it never returns JSON schematics as well as tool information, which we will delve into soon, and much more information is being fed to the AI-agent, which the user should not be given. Now, the user input as well as the embedded information for the AI-agent has been connected, and the flow continues, in which any relevant tools are found, based on the current user input. An example of such a tool could be as follows:

```
1 "type": "function",
2 "function": {
3   'name': 'get_customer_by_email',
4   'description': 'Find a customer and their info from the customer table
5   by their email.',
6   "parameters": {
7     "type": "object",
8     "properties": {
9       "email": {"type": "string", "description": "The email that
10       wanted customer uses."}
11     },
12     "required": ["email"]
13   }
14 }
```

Code Snippet 5.1: Agent tool: get_customer_by_email

The tool is a JSON schematic, packaging information easily understood and utilised by the AI-agent. Especially the *name* and *description* carry weight, as the name aspect is what is later used to match with the function of the same name. The description is how the AI-agent understands if it should call said tool. So if the user input in natural language matches the meaning of the description of the tool, the AI-agent will be able to call said tool. Whenever any tool is called upon, our matching function matches the tool's name to the appropriate function, and the results are given to the AI-agent.

The tool in Code Snippet 5.1 matches a function containing a simple Read query, whose returned data is translated into the JSON format, to better parse information to the AI-agent.

The AI-agent now continuously calls any tools and therefore functions, related to the original user input as well as embedded knowledge. The AI-agent reads the results of the tool calls before deciding if it needs to act on that information by calling more tools or by writing a response to the user. An example of this would be if the AI-agent wanted to look up whether an email was already in use by another user. The AAI can ask for a given email and input it into the JSON schema (Code Snippet 5.1), and so the function call can result in a list of user information or an empty list. The AI-agent has then, through its embedded rules, been instructed to ask for any necessary information for its tool calls.

One of the AI-agent's central responsibilities is not only to interact with the user but also to recommend suitable dates for cleaning appointments. These recommendations are based on three factors:

1. Calendar availability
2. Weather conditions
3. User-type

The calendar and weather components, described in Section 5.2.2, are designed as general-purpose inputs that support all recommendations. In contrast, the user-type dimension is meant to tailor both the agent's language and its suggestions to the individual user. As outlined in Section 4.1.1 and Section 5.1.1, we defined a set of user-types with characteristic patterns that the AI-agent can match against. The AI-agent classifies the user into one of these types and uses this information alongside the user's inputs, relevant tool outputs, and the ongoing conversation history. Together, these elements form the agent's working knowledge base, from which all final recommendations are drawn. However, by design, the agent's output is restricted to natural language, ensuring that the interaction remains clear and user-friendly.

5.2.4 Frontend

The System Interface

The frontend of the second prototype is implemented using Flask as the web framework, and Flask functions as the primary system interface between user-facing components and the backend. Compared to the first prototype, where the frontend only supported digitalised administrative tasks, this iteration places the system in a more active coordinating role. This change reflects the redefined system boundaries introduced earlier in Figure 4.1, where the server and AI-agent are positioned as the primary actors in the workflow rather than supporting elements.

From an OOAD-perspective, interfaces define behavioural boundaries that specify what a system offers to the outside world, while hiding how those behaviours are implemented internally (Mathiassen 2018, p. 169). This understanding guided the design of the system interface. Instead of embedding logic directly in the frontend, Flask exposes a set of clearly defined routes and APIs that act as controlled entry points to the system's functionality. These routes decide the responsibility to the booking application, the AI-agent, and the backend services, ensuring that the decision-making remains inside the backend.

This approach is closely connected to the system context described earlier in Section 4.1. With the administrative employee removed as an intermediary actor, the system itself becomes responsible for interpreting user input, coordinating bookings, and handling changes. The system interface allows this shift by acting as a stable access layer. It does not decide what should happen, but it ensures that every interaction follows the same internal logic and rules. Whether a user modifies an appointment through natural language or through a form-based interaction, the request passes through the same interface and is executed under identical conditions.

In relation to the architecture described in Section 5.2.1, the frontend corresponds to the `Component Interface` layer. Its role is limited to invoking system behaviour through the routes and APIs, while all the reasoning and automation are handled within the `Component Backend` and the `Component AI-agent`. These components, in turn, rely on the external services such as our database, calendar, weather API, and OpenAI's language model. By keeping decision-making and automation outside the frontend, we maintain the boundaries between components. Several of the requirements identified earlier in Section 5.1.2 are directly supported by this design. Functionally, it allows conversation-driven and manual booking flows to coexist without duplicating the logic. From a non-functional perspective, it improves maintainability and scalability by making it possible to adjust booking rules, AI behaviour, or other external integrations, without interfering with the interface layer. Finally, the system interface plays an important role in shaping the overall interaction, even though it remains largely invisible to the user. By exposing a limited set of actions that reflect user intentions rather than technical processes, the interface reduces unnecessary complexity. Users are not required to understand how the system is structured or where the decisions are made. Instead, the interface acts as a contract between the frontend and the automated parts of the system, allowing the interaction to be flexible while the system behaviour stays consistent and predictable.

The User Interface

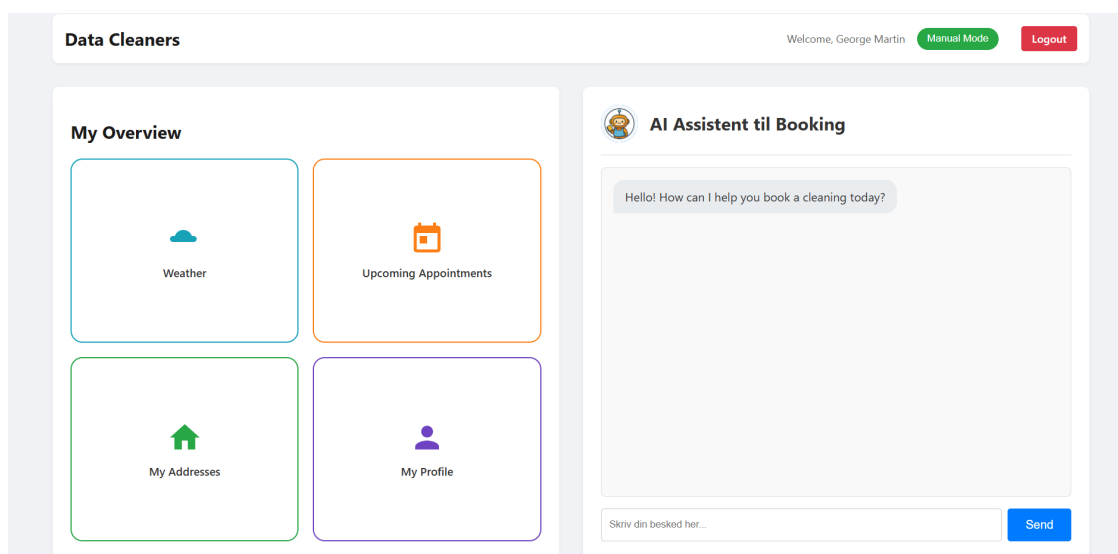


Figure 5.9: Frontend

UIs are described by Mathiassen 2018 as being organised around user tasks rather than internal system structure, so that interaction reflects how users understand and work with the system in practice (Mathiassen 2018, pp. 153–155). This principle guided the design of the frontend UI, which is organised as a frontend implemented in HTML and styled using CSS. The dynamic behaviour, such as the widget interaction, modal handling, and AI-agent communication, is managed through JavaScript. The frontend is divided into two primary areas. On the left, four widgets provides access to the system’s main features, consisting of weather information, upcoming appointments, registered addresses, and user profile data (see Figure 5.9). When selected, their content is loaded into a modal overlay, allowing the user to access detailed information without losing the surrounding context of the frontend. This supports task-oriented interaction and reduces unnecessary navigation. On the right side of the interface, the AI-agent is presented as a central interaction point (see Figure 5.9). Messages entered by the user are sent to the backend through the `/api/chat` endpoint, where they are processed by the AI-agent before returning a response. This interaction style reflects the revised system boundaries identified in the revisited problem domain (see Section 2). The AI-agent exposes the system behaviour in a conversational form, while the underlying logic, such as booking rules and database queries, remains hidden from the user. In addition to the frontend, the system provides a manual booking interface implemented in HTML with supporting logic in JavaScript. This interface offers a more traditional form and calendar-based model, allowing the user to manually create, update, and delete their appointments (see Figure 5.9). Although the style of interaction differs from the conversational aspect of the AI-agent, the components remain consistent across interfaces. For instance, the same calendar component defined in `calendar_widget.js` is reused in both the frontend and manual mode, which ensures that appointments are presented in the same way regardless of how they are accessed, see Figure 5.6 and 5.10. A mode toggle allows users to switch between the AI-assisted frontend and the manual booking interface (see Figure 5.9). The toggle does not alter how bookings are processed or stored. Instead, it redirects users between Flask routes that connect to the same backend services and data. This avoids duplicating logic or state while still supporting different preferences of user interaction.

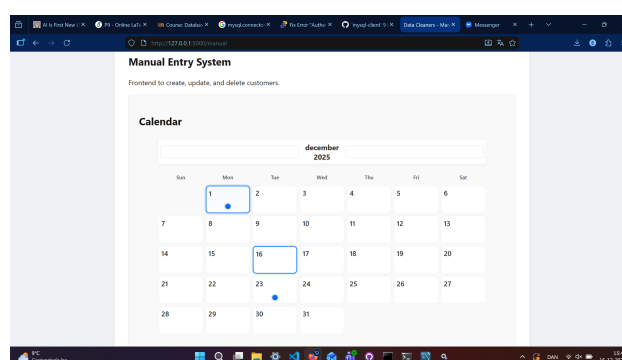


Figure 5.10: Manual Calendar

5.2.5 Subconclusion

The design and implementation choices of the second prototype successfully restructured the system around the customer and automation, moving the core responsibilities from administrative staff to the booking application and AI-agent (Figure ref5.3). The use of specifications and diagrams (Figure 5.1 and 5.2) ensured that the developed features directly address the essential functional and non-functional requirements. This process culminated in a system architecture capable of using external data (weather, calendar) to autonomously generate and manage complex booking schedules.

With the implementation of the core features, the immediate question shifts from how the system works to how well the system works. The following section, evaluation and discussion, will therefore analyse the implemented solution in regards to the problem statement, through the use of heuristics in expert evaluations.

6 Evaluation & Reflection

This chapter evaluates the second prototype in relation to the problem statement in Section 2.3. The evaluation combines feedback from the case partnership and a *Heuristic Evaluation* (HE) based on Jakob Nielsen’s usability heuristics (Jakob Nielsen 2024), allowing the system to be assessed from our stakeholder’s perspective, as well as through hands-on user testing. The discussion builds on these findings by reflecting on how our design choices affect automation, user understanding, and cognitive load. Instead of focusing only on the individual usability, the chapter considers whether the overall interaction supports efficient task completion and whether a conversational, AI-supported interface is the right approach for this type of booking system.

6.1 Case partnership feedback

The case partnership gave us a valuable perspective throughout the project, despite the company itself being hypothetical. The feedback from our case partner was collected through written email evaluations, which are cited below and included in Appendix B. While the hypothetical nature of the case limited the extent to which real constraints could shape the concept and design, the collaboration still gave us a meaningful point of reference and evaluation. In their written feedback, they stated that they were “impressed by how far you have come with your solution” and highlighted the “intuitive use of the AI Booking Assistant” within a “clear user interface”. They also noted that the *Retrieval-Augmented Generation* (RAG)-based use of customer data “adds a personal touch” and “provides a solid foundation for personalisation”. The feedback took the form of an evaluation rather than continuous co-design, with comments only confirming already implemented decisions. The concrete point of critique concerned interface consistency, where we “could benefit from being more consistent in [our] choice of language” due to the mixed use of Danish and English responses of the AI-agent. Although a minor problem, this highlighted an area for improvement in future iterations. Regarding collaboration, the group was described as showing a “high level of engagement” and an “impressive degree of independence”, delivering “rapid results”.

While the email-based feedback from the case partner gave useful validation of the solution, it mainly reflected a stakeholder perspective rather than direct use of the system. To further investigate this, we carried out our own evaluation utilising Jakob Nielsen’s heuristics, focusing on how real users interacted with the prototype in practice (Jakob Nielsen 2024). This made it possible to identify usability issues and strengths that were not visible through case-partner feedback alone, and it forms the basis for the following evaluation section.

6.2 Heuristic Evaluation

6.2.1 How we used the Method

To evaluate the usability, we applied the method of HE based on Jakob Nielsen's Ten Usability Heuristics (Jakob Nielsen 2024). We sought inspiration in the guidelines on "How to Conduct a Heuristic Evaluation", by Moran and Morgan 2023, which provided ideas on how to structure our own framework for identifying usability issues in the conversation flow with the AI-agent as well as the user interface in general. The purpose of this evaluation was to reveal both obvious and subtle usability problems that could affect the customer's ability to interact with our interface successfully and save time when using the website. This HE was performed by five individual experts, who gave feedback on our booking system interface. These experts consist of fellow students of Digitalisation and Application Development in the ninth semester, as they possess the same level of professional and theoretical know-how of how the technical aspects work for both development and design. The evaluation was conducted by two group members evaluating each expert, to ensure independent observations and to increase the breadth of identified issues. This was followed up by a group discussion on which findings and specific heuristics to highlight in our analysis. Each expert was given specific tasks, one at a time, to examine the prototype to test what we deemed as core functionalities, with our problem statement in mind. One group member was responsible for reading the tasks aloud and making sure the experts understood the given task; the other noted potential transgressions of the heuristics, such as "Visibility of System Status", "Match Between System and the Real World", or "Consistency and Standards", accompanied by a brief description of the observed action. Finally, we compiled our results into a workbook (see Appendix C), one for each expert. This workbook was used to sort the information gathered from each evaluation into each heuristic, where applicable.

Based on this structured evaluation process, we now turn to the results themselves. In the following section, we present the key usability issues identified by the experts and analyse how these findings impact the interaction flow of our prototype and its ability to support automated booking procedures in practice.

In the rest of this chapter, we go through the observed results of the tests, before prioritising them based on severity, and then discuss how the application could be improved to lessen or remove these limitations.

6.2.2 Identified Usability Issues

We have identified different categories of usability issues from our evaluation, which will be described in the following section.

Although all five experts successfully navigated through the tasks using both the conversational and graphical interfaces, one participant was unable to complete the final task due to an unrecoverable system error. Consequently, though the expert completed the session, the data for that specific task remains incomplete.

Technical Ability

The AI-agent could generally respond as intended, keep the conversation focused on appointment booking, and request the necessary information as intended (see Section 5.2.3). By double-checking the database and list of tools used by the AI-agent after each test, it was confirmed that the AI-agent also performed the database operations the expert asked for.

AI expectations

Despite the AI-agent's technical consistency and its ability to assist in all tasks given, we observed multiple cases of hesitation from the experts when interacting with the AI-agent. These cases of hesitation came, in part, from an uncertainty that probably stems from CUIs belonging to a different paradigm of user interaction than GUIs. Two experts gravitated to the GUI elements of the homepage when starting the first task, despite being explicitly told to register their address using the AI-agent. We observed several aspects of how the experts learned to use the AI-agent during the conversation.

Some would try to respond to the AI-agent's messages with short informal sentences, and some took the time to respond with more formal answers, giving more information than may have been needed, as seen in figure 6.1. Here, the expert mentioned the date of their booking as both "tomorrow" and "the 10.12.2025". One of these should be enough, and specifying the year should be unnecessary, for the AAI to perform the booking, but that was not presumed by the expert.

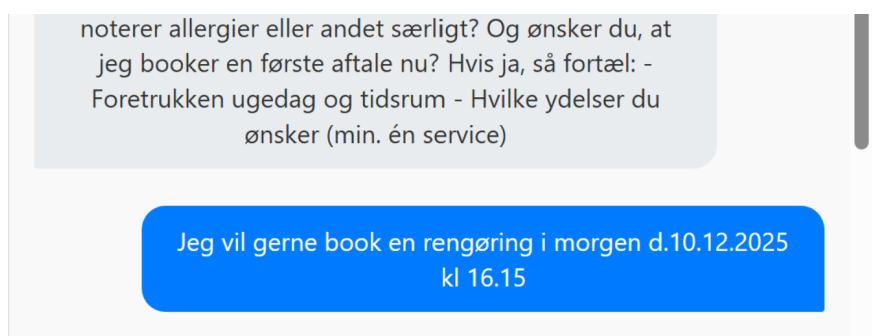


Figure 6.1: Screenshot of an expert giving detailed information

One recurring finding among the experts was confusion about what kind of messages could be written to the AI-agent. In four evaluations, it was not clear to the experts what kind of chat input the AI-agent could respond to. In three of those evaluations, it was also unclear what kind of actions, like address registration or appointment booking, the AI-agent could help perform.

For those four evaluations, it was also not clear what language one should use to make the AI-agent respond in the chat, a sentiment which was repeated by our case-partners in Section 6.1.

System visibility

When an action has been performed by the AI-agent, it says so in its response to the user. An example of this would be that the user's address has been registered into the database, in which it will respond with: *"Thanks! Your address is now registered [...]"* before moving on to suggest that it recommends a date to book an appointment. However, the experts were sometimes still left with some doubt whether, in this case, the address had been added to their account, thus the task had been completed. Figure 6.2 shows an example of this, where the wording of *"We will get that done at [address]. [...]"* left the expert unsure and made them ask the AAI to confirm.

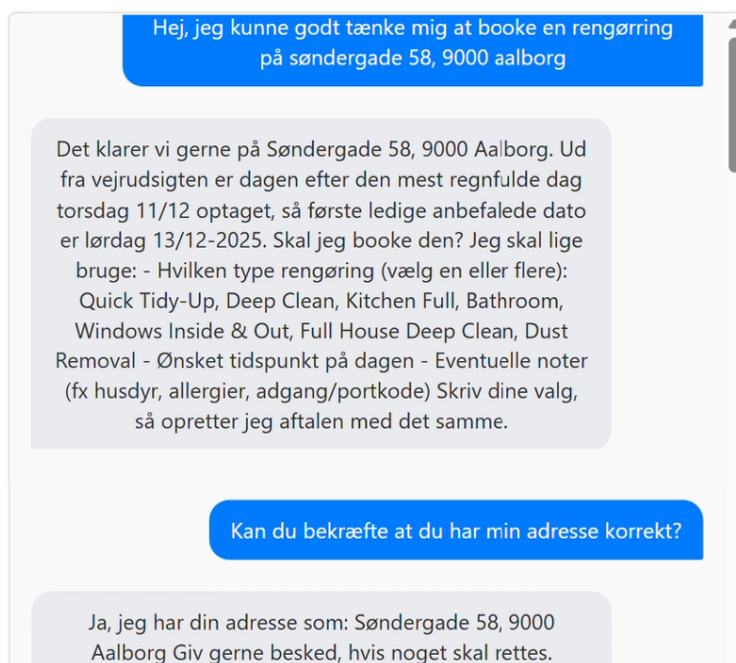


Figure 6.2: Screenshot of AAI being unclear about address registration

Two experts expressed being worried that the AI-agent was hallucinating, and three thought that it may not be capable of performing the actions it was claiming to do, leading to some level of mistrust. Related to mistrust, one of the experts described their past frustrations with other AI-agents, especially customer service-oriented AI-agents, when our interface was presented to them. They had past experiences with company AI-agents failing their tasks, which they expressed was part of the reason why they were hesitant to use our AI-agent.

Errors

Across all experts, the wait time between AI-agent responses had them wondering what was happening behind the scenes, or if they might interrupt something by looking through the widgets. During the first evaluation, the expert experienced an error with the AI-agent. It seemingly forgot the whole chat history and acted as if the conversation had started over. This happened as the expert reloaded the page while the AI-agent was working on a response. The AI-agent still responded correctly while the rest of the chat history had visibly disappeared from the interface. The AI-agent was now just asking the expert to confirm their booking. Figure 6.3 shows the user trying to confirm the information within the booking that is about to be confirmed, but the AI-agent suddenly asks for all the information previously provided.

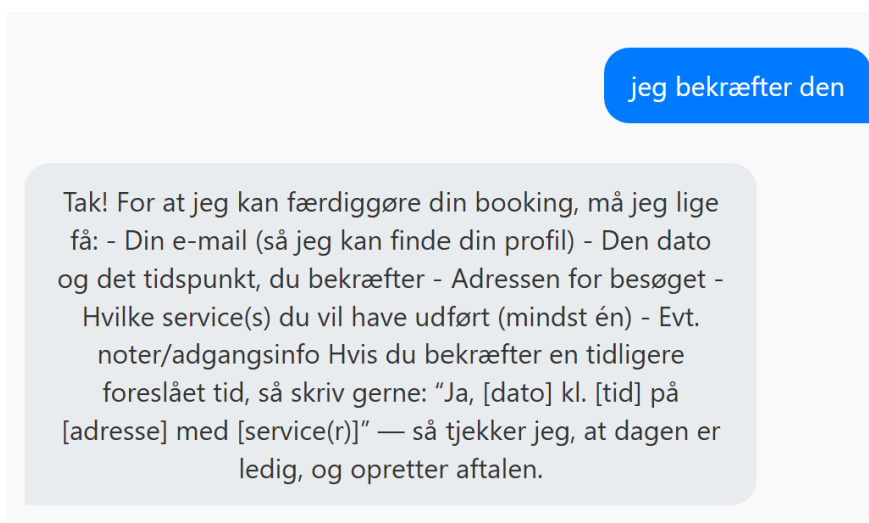


Figure 6.3: Screenshot of the AI-agent forgetting the context of the conversation

Information presentation

Figure 6.3 shows another problem that was experienced by some of our experts, namely, how information is presented in the conversation. Its tendency to ask for all the needed information at once and how those tasks are presented as an unformatted text message was something experts deemed as overwhelming.

Their ability to easily understand the messages received by the AI-agent, to respond accordingly to the messages from the AI-agent, and to achieve the goal of booking an appointment was ultimately made more difficult by these factors.

We mapped our different kinds of usability issues, discovered in our evaluation, on a coordinate system, with frequency and impact on the products ability to solve our problem statement.

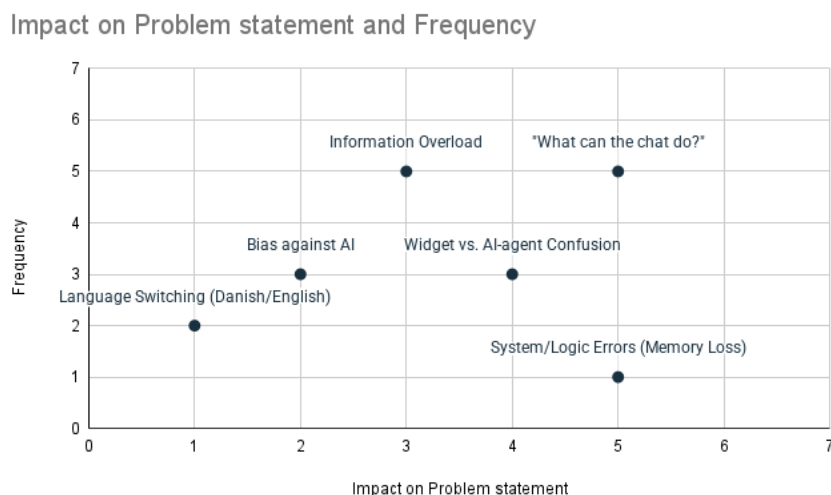


Figure 6.4: Diagram mapping usability issues along frequency and impact on problem formulation

The results that place closest to the top right of figure 6.4 become the most important to address, as these are both the most detrimental to our project, and are detrimental the most often. This analysis helped decide what features would be looked at for future improvement.

6.2.3 Prioritising Heuristics

The results in *AI expectations* relate to the *Consistency and Standards* heuristic, because the concern is about a mismatch between the system's functionality and the experts' expectations about it. Our application does, in some ways, not align with the expectations of usage that the experts came into the evaluation with.

The System visibility and *Errors* sections relate to the *Visibility of System Status* heuristic. If an expert is not sure what information has been registered, only with the AI-agent's feedback, it is an issue because it then becomes much less clear about what actions to take next. Because it is sometimes not clearly communicated what error is or is not happening, it likewise becomes difficult to use the application correctly.

Information presentation describes results relating to *Aesthetic and Minimalist Design*. When the AI-agent asks for or gives a lot of information all at once, information that might not be needed immediately, it makes it more difficult for the user to focus on and perform their goal.

As a result of the identified usability issues, we felt a need to prioritise certain heuristics to dedicate ourselves to looking into the valuable results they provided. The selection of our heuristics has two criteria:

1. Influence on the automation of pre-existing manual booking procedures
2. Looking at which of these selected heuristics occurred more often.

To have a solid reasoning behind our choices, we gave each unique heuristic a so-called 'severity rank'. Nielsen states that these ranks "... can be used to allocate the most resources to fix the most serious problems and can also provide a rough estimate of the need for additional usability efforts (Jakob Nielsen 1994)". With this in mind, we used it as a structural guide to organise our findings and selected the usability issues that, by looking at their severity ranks, had the greatest impact on automating the booking flow for our users. The selective output for the chosen heuristics is shown in the list below:

1. *Visibility of System Status*
4. *Consistency and Standards*
8. *Aesthetic and Minimalist Design*

6.2.4 Justifying the chosen heuristics

Visibility of System Status received the highest severity ranking due to recurring issues related to unclear system feedback and uncertainty during interaction. Several experts experienced confusion when interacting with both the AI-agent and interface widgets(described in Section 6.2.2), which disrupted the natural progression of user actions. Although widget interaction was not central to the booking-by-chat flow, this uncertainty caused noticeable hesitation and irritation, indicating a lack of clarity regarding the system's current state.

In contrast, *Consistency and Standards* emerged as a significant source of unnecessary cognitive effort. Variations in language, tone, and level of detail, particularly within the AI-agent's messages, caused experts to hesitate and occasionally re-read information or take additional interaction steps. These inconsistencies challenged users' expectations and slowed down the booking flow, despite the system's technical ability to complete the tasks. In several instances, users interacted with interface widgets instead of the AI-agent, suggesting that the visual prominence and placement of interface elements did not clearly communicate the intended primary interaction channel.

Finally, *Aesthetic and Minimalist Design* was prioritised due to its influence on both clarity and efficiency. The experts repeatedly reported that long and frequent AI-agent messages made essential information difficult to extract.

By highlighting these three heuristics, we identify the usability issues that most strongly affected users' interaction flow and their ability to efficiently complete booking tasks within the prototype. As mentioned in Section 2.3, this desired extra value more specifically means to put less cognitive load on the users by looking at how many elements one has to consider when solving a given task and whether this leads to a more efficient attempt to solve that given task.

6.2.5 Suggestions for improvements

Based on the prioritised heuristics, several improvement opportunities can be identified that may enhance the usability of the prototype, and thereby, its ability to increase automation.

In relation to *Visibility of System Status* and the *System visibility* issues, addressing uncertainty regarding system responses may strengthen users' understanding of the booking process. Clearer and more concise feedback following user actions, including information about *Errors*, could help users maintain awareness of system progress, thereby reducing hesitation and interruptions in the interaction flow. While working on a AI-agent response, one expert expressed appreciation towards the interface displaying "AI Typing", to indicate that things are happening behind the scenes. The "AI Typing"-message that is displayed, while the AI-agent is working on a response, could be expanded to be more specific about the actions being performed by the AI. If, for example, a request to register an address had the chat UI show "Looking up addresses" into "Saving your address" into "AI Typing", it should more transparently show what is happening behind the scenes and help keep the user from wondering what is happening and whether an error has occurred during the time between responses. This should also help communicate what the AI-agent is capable of, so the user can learn, during the conversation, what to expect from the AI-agent. A hopeful outcome of this would be to heighten the trust that the user has in the system.

Regarding *Consistency and Standards* and *AI expectations*, establishing a more uniform communication style across the AI-agent and interface elements may reduce the need for users to reinterpret information throughout the interaction. Aligning terminology, tone, and message structure with user expectations could support smoother task execution and limit unnecessary navigation or clarification steps.

The experts' use of the widgets seemed to suggest that having them compete with the AI-agent interface, visually, made it more difficult to know where actions like "See your new booking" should be performed. In one case, an expert wanted to register an address from the "My Addresses" widget, rather than using the AI-agent as intended. In another case, an expert checked their new booking by asking the AI-agent about it, rather than looking at the "My Calender" widget as intended.

As Chat-GPT is the leading application in AI-agent usage, Chat-GPT's website ([OpenAI, n.d.](#)) might be the kind of interface that people expect from AI-agents. In that interface, only the edges have buttons unrelated to the interaction with the AI-agent, while the entire centre of the screen is dedicated to the text input field, the AI-agent's response, and AI-agent options. Taking inspiration from this interface to make our own more AI-agent-centred should help guide the user towards interacting with it, and make it clear that its supposed to be the primary tool. Webshops that use AI-agents often have them in a separate window floating over the rest of the page, in the bottom right corner, more separate from the rest of the page, and are usually only able to provide information. In line with what one expert expressed, this could set an expectation in users that our AI-agent is not meant to perform actions on behalf of the user. Jakob Nielsen argues that the interaction with and use of generative AI represents a paradigm shift separate from the GUI applications that we are used to ([Jacob Nielsen 2023a](#)). We know that we are making an interface that does not behave as most users have come to learn and expect a booking website to behave. Given this, we would try to lessen misunderstanding about how to interact with the AI-agent by giving more hints and instructions on what actions the AI-agent can provide. Make the AI-agent take up the centre of the screen, and rewrite the AI-agent's initial message to be more specific about its abilities and goals. Changing "How can I help you book a cleaning today? You can say 'Book a cleaning for a new customer'" to a more comprehensive message about its different categories of available actions. These changes could explicitly and implicitly communicate to the user that the AI-agent is the primary way of performing actions like booking. The widgets could be moved to a vertical icon menu on the side of the screen.

The insights derived from this heuristic are closely related to the results from *Information presentation* and the last of our chosen three heuristics, *Aesthetic and Minimalist Design*, which suggests that reducing information overload and clarifying visual priorities may contribute to more efficient task completion. By emphasising the AI-agent as the primary interaction channel and limiting competing interface elements, the prototype may place less cognitive demand on users and better support time-efficient booking behaviour. As shown in Figure 6.3, the AI-agent can ask for a lot of information at once. At least 9 pieces of information, in this case. As one expert also suggested, we could help reduce information overload, and clarify what information should be prioritised, by formatting the text within a message better, splitting one message up into multiple. Formatting each piece of information into a list should make it more readable. When answering a message, the experts would sometimes reread the message to ensure they had written all the information being asked for. Bolding specific words like "{address}" and "{date}", that the user is being asked to provide could make it easier to double-check. Only asking for one piece of information per message should also help to not overwhelm users with information about what actions they should perform. This would also result in more prompts happening to ChatGPT in total, which is a tradeoff. These suggestions for improvement highlight how relatively targeted design adjustments may contribute to increased user understanding and more efficient task completion, thereby supporting the overall goal of automating the booking process.

6.3 Reflections on the Findings

This section reflects on the extent to which the proposed improvement suggestions may enhance the prototype’s ability to deliver meaningful conversational automation. Rather than reiterating specific usability issues, the discussion considers whether the suggested changes are sufficient to reduce cognitive load and support efficient task completion. Several of the usability challenges identified during the evaluation were not directly related to whether tasks could be completed, but rather to the effort required to understand and engage with the AI-supported interaction. To examine this aspect more explicitly, we consider the system from a cognitive load perspective, focusing on how the design of the conversational interface influences users’ experience of the automated booking process.

6.3.1 Balancing Automation and Cognitive Effort

Cognitive load refers to the amount of mental effort being used for any given task. The higher the cognitive load, the more difficult the task is, or the bigger toll it will have on the one performing the task (Sweller 2020). The concept of cognitive load can be viewed as a mental bandwidth, which when thought into our case, would be applicable to the effort required to perform any interaction with the AAI, e.g. booking an appointment or editing existing ones. Cognitive load is added when the user has to decide on their next action, and when they have to remember the answers they get (e.g. if the UI doesn’t allow one to go back in the chat history). The aim of utilising AAI to replace the manual process of contact, booking and finding suitable times for appointments, is to ensure that the user’s cognitive load is smaller overall throughout the process of their task. Current research indicates that AI, especially in the context of AI-agents with integrated functions for the task at hand, can lower cognitive load (Schmidhuber, Schlogl, and Ploder 2021). An issue with our goal of reducing cognitive load for our users has, as mentioned in Section 6.2.5, been influenced negatively, as some of our experts experienced lengthened responses and increased confusion from a lack of introduction to the AAI as the one-size-fits-all solution of our system.

Beyond general mental effort, our evaluation also points to a challenge related to how users are expected to express what they want to do. Jakob Nielsen describes this as an *articulation barrier*, where text-based interfaces place a higher demand on users because explaining intentions in words can be harder than recognising or selecting actions (Jacob Nielsen 2023b). Although the AI-agent is capable of handling complex requests, the experts often turn to the widgets when dealing with more routine tasks. This suggests that for everyday coordination, a purely conversational interface can feel like extra work rather than a helping hand. In the context of Unit Cleaning, this indicates that AI may work best as part of a hybrid solution. The AI-agent can support tasks that require explanation or flexibility, such as special requests, while widgets offer simple and familiar ways to confirm actions or view information. If we were showing users what they could do instead of asking them to phrase everything themselves, the system could reduce effort and make the interaction feel more intuitive and straightforward.

6.3.2 Efficiency, reliability and User Trust

When viewed through the lens of cognitive load, task efficiency becomes less about the speed and more about how much mental effort the system asks of the user while a task is being carried out. In this sense, efficiency is not just a question of doing things faster, but of how well the automation reduces decision-making, uncertainty, and the need to remember one's details during interaction with the AI-agent. Our evaluation suggests that the automation supports this goal to some extent, but not without trade-offs. On the positive side, the AI-agent replaces a process that would otherwise involve manual contact, coordination, and booking. This removes several cognitively demanding steps from the user's task flow and reduces the need to actively manage appointments. At the same time, expert feedback pointed to moments where longer responses and an unclear introduction to the AI-agent created confusion. In these cases, the system appeared to add mental effort rather than relieve it, which shows that automation alone does not guarantee lower cognitive load unless the interaction is shaped to achieve that aspect specifically.

This highlights a central tension in the design of automation. On the one hand, providing explanatory feedback, such as telling users what the AI is doing or clarifying what happens behind the scenes, can support understanding of the task flow and reduce uncertainty. That kind of transparency can make the system feel more predictable and trustworthy. On the other hand, these explanations also extend the conversation and may slow down the user's sense of progress, which can undermine the very efficiency automation aims to create. A more action-oriented dialogue that keeps responses short and focused can feel faster and more streamlined, but it also risks ignoring differences in how users interpret information and how much guidance they need during interaction. Therefore, we argue that efficiency cannot be understood purely through quantitative measures, such as the number of interaction cycles. While fewer conversational turns can suggest faster task completion, our evaluation indicates that efficiency is just as dependent on qualitative aspects of the interaction. These include the user's sense of control, their trust in the system, and the level of frustration they experience along the way. Experts who found it difficult to understand the role and scope of the AI-agent also reported lower confidence in the system, which directly influenced how useful they perceived the automation to be.

One of the clearer limitations revealed through the evaluation was reliability. In Unit Cleaning's current, more manual setup, a human administrator acts as a kind of shared memory, keeping track of details and making sure progress is not lost when something unexpected happens. In the prototype, we saw moments where the AI-agent lost track of the conversation, see Figure 6.3, which immediately disrupted the interaction. When this happened, the issue was not just that the task took longer, but that users became unsure whether the system could be trusted at all. This suggests that conversational automation is still sensitive to breakdowns in a way that manual processes are not. The AI-agent can handle booking coordination when everything runs smoothly, but its usefulness quickly drops if it cannot reliably maintain context or recover from errors. In those situations, users are forced to repeat themselves or retrace steps, which adds frustration and increases cognitive effort. Rather than pointing to full automation as the obvious next step, the prototype highlights the importance of support mechanisms, such as saving progress automatically or allowing a human to step in when needed. These elements appear necessary if an AI-supported booking system is to feel dependable enough for everyday use in an organisation like Unit Cleaning.

The prototype also revealed a clear reliability gap. In Unit Cleaning’s current setup, an administrator employee has shared memory, making sure nothing is lost if something goes wrong. When the AI-agent forgot the conversation context during testing, the issue was not just a technical hiccup. It immediately broke user trust. This highlights a limitation of the current automation. If an AI system fails mid-task, the cost to the user is higher than in a manual process, because the work has to be repeated. In that sense, automation can actually increase cognitive load instead of reducing it. To make this kind of system viable, it would need stronger safeguards, such as automatic state saving or clear handover to an actual administrative employee if something breaks.

These findings suggest that although the automation shows clear potential to reduce cognitive load and support more efficient task completion, it does not yet constitute a fully optimised solution. The current implementation succeeds in automating key elements of the booking process, but it remains inconsistent in how system status and intent are communicated to the user. At times, this shifts cognitive effort rather than reducing it. Task efficiency, therefore, emerges not as a single, easily measurable outcome but as a balance between reduced cognitive load, conversational design choices, and the user’s trust in the system. Future iterations could focus on simplifying the interaction while making system behaviour more transparent, so that automation consistently alleviates cognitive effort instead of redistributing it elsewhere in the interaction.

Future Perspectives

The evaluation also raises questions about how this kind of system fits into current practice. While the prototype shows that an AI-supported solution *can* handle large parts of booking coordination, the user feedback suggests that trust and shared understanding still need time to develop. Several experts hesitated during interaction, not because the system broke, but because it was not always clear what the system was doing or responsible for. This suggests that conversational automation is more than just a technical change, it also affects how work is delegated between humans and systems. Seen this way, the prototype works best as an exploration of what is possible right now rather than as a finished solution. In a sense, it could represent a first step toward more advanced automation in this context, where future iterations could build on the idea by improving clarity, reliability, and alignment with user expectations. Rather than arguing that a AI-agent is already the best solution for Unit Cleaning, the prototype helps clarify when and how such a system might realistically become useful.

Our different components, shown in the component diagram in figure 5.3, are made to function relatively independently of each other, which will make the components easier to reuse for other businesses with online browser-based booking. The chatbot component can be reused, given that some time is spent customising it with information and rules relevant to the given business and role of the agent. The list of tools would also need to be modified to fit the relevant database and wanted functions. They would also need their own DMI API key and Chat-GPT API key. Our database is also designed with modularity in mind. By making the tables atomic, we allow for parts of the database to be reused for other organisations, that for example may need to store the same information on users, but needs different information. The frontend components of the chatbot make up a self-contained chat window which could be reused together with the AAI component, assuming the company website can dedicate half a browser page to the chatbot interface.

If they want to change colours or other visual elements, some modifications are also needed. These factors mean that our AAI chatbot could not be a completely off-the-shelf product for companies to pick up and implement. Some time and expertise would be required. The tradeoff is that the agent is more customisable and able to better fit the company's wishes and goals with a chatbot implementation. Because chatbots like ours are made to resemble a conversation with human sellers or customer support, organisations might utilise the ability to personalize the chatbot with vocabulary, accents or actions that fit their image. Likewise, different organisations can have different wants and needs for what actions the AAI should take and when. This can be specified to fit their pre-existing workflow, minimizing what organisational change the AAI product expects of the organisation.

Scope and Limitations of the Project

We framed the project as an investigation into the potential and limitations of conversational automation rather than an attempt to deliver a finished or fully validated solution. While the second prototype demonstrates how an AI-agent can take over parts of the booking workflow, the evaluation remains limited in scope. The prototype was tested through expert-based HEs rather than long-term use by real customers, and the case itself is hypothetical, which restricts the extent to which the organisational constraints could be fully explored. As a result, the findings primarily reflect interaction-level challenges such as cognitive load, system visibility, and user trust, rather than outcomes such as time saved or reduced administrative needs. This limitation is consistent with the exploratory nature of the semester project. The value of our investigation lies in identifying where automation supports user interaction and where it introduces friction, rather than in claiming measurable efficiency gains. We find that these boundaries are important to acknowledge, as they shape how the proposed product should be interpreted.

7 Conclusion

This project has focused on the development of an interactive booking system that addresses an organisational challenge related to manual coordination and administration in service-based appointment booking. This brings us back to the problem statement that guided the development of the project.

How can an AI-supported conversational booking system be designed to investigate the potential and limitations of automating appointment coordination in Unit Cleaning's context?

Starting from a hypothetical but realistic case context, we explored how a system could automate existing booking practices. Rather than approaching the problem as a matter of identifying new user needs, the project was an iterative development process, where insights from the first prototype were reflected in the design and implementation of the second. Through this process, we identified that the primary limitation of the initial solution was its focus on digitalisation rather than automation. Although the first prototype reduced some manual effort, it still relied heavily on administrative involvement. This realisation led us to redefine the problem domain and system boundaries, shifting responsibility from administrative staff to the system itself. The second prototype was therefore developed with a stronger emphasis on automated coordination, positioning the customer as the primary user and allowing the system to take on the tasks of scheduling, adjustment of appointments, and contextual decisions based on actual calendar and weather data.

A central design decision in this iteration was the introduction of an AI-agent as the main element of interaction. This choice was not based on empirically identified user demands, but on the need to explore how conversational, agent-based interaction could support automation while remaining flexible across different contexts and user scenarios. User stories were used as analytical design tools to explore interaction patterns and different user contexts to establish the requirements for the system. They helped shape the system's behaviour and supported the development of a solution that could accommodate different levels of user interaction and personalised preferences. The design and implementation of the system demonstrate how our interactive solution was developed systematically within an organisational context. The resulting prototype integrates a modular backend, an AI-agent capable of performing actions across system components, and a UI that provides conversational interaction with the AI-agent.

Working with and building upon an AI-agent also contributed significantly to the learning outcomes of the project. Unlike traditional software components, the AI-agent introduced a degree of uncertainty and unpredictable behaviour, which required careful consideration of system boundaries, transparency, and responsibility. This reinforced our understanding that developing AI-supported systems is not purely about the technical tasks, but also design, communication and organisational challenges. Our decisions evolved throughout the process in terms of what to automate, how to structure the interaction, and how to support user trust.

The evaluation showed that the prototype is capable of supporting the intended booking tasks and that the interaction flow works on a functional level. At the same time, the HE revealed challenges related to trust, clarity, and cognitive load, especially in the conversational interaction with the AI-agent. While automation reduces parts of the manual coordination work, the findings suggest that this alone is not enough to ensure smooth UX. Instead, the perceived value of the system depends strongly on how clearly it communicates its role, actions, and current state. When feedback was unclear or context was lost, users' confidence dropped, and the mental effort required increased. This showed that the evaluation was not only about whether the system worked, but also about when and how automation can support users in practice.

In conclusion, the project demonstrates how an AI-supported, conversational booking system can be developed through iterative refinement to address an organisational administrative problem. Our work aims to meet the semester expectations by systematically examining an organisational challenge, designing and implementing an interactive system to automate manual booking practices, and investigating how the solution could create value. At the same time, the project aims to meet the learning objectives by combining problem formulation, system development, evaluation, and reflection on practical implementation challenges. While the prototype is not a finished product, it provides an insight into how interactive, AI-supported systems can be developed and implemented in an organisational context.

Bibliography

- Beck, Kent, et al. 2001a. *Manifesto for Agile Software Development*. <https://agilemanifesto.org>. Last visited: 10-12-2025.
- . 2001b. *Principles behind the Agile Manifesto*. <https://agilemanifesto.org/principles.html>. Last visited: 12-12-2025.
- Boehm, Barry, and Richard Turner. 2003. “Observations on Balancing Discipline and Agility.” In *Proceedings of the 26th International Conference on Software Engineering*, 718–719.
- Chowdhury, Rakibul Hasan. 2024. “AI-driven business analytics for operational efficiency.” *World Journal of Advanced Engineering Technology and Sciences (WJAETS)* 12 (02): 535–543. <https://doi.org/10.30574/wjaets.2024.12.2.0329>. <https://doi.org/10.30574/wjaets.2024.12.2.0329>.
- Cohn, Mike. 2004. *User Stories Applied: For Agile Software Development*. Boston: Addison-Wesley Professional. ISBN: 9780321205681.
- Cursor. n.d. *Features - The best way to build software*. <https://cursor.com/features>. Last visited: 10-12-2025.
- Elmasri, Ramez., and Shamkant B. Navathe. 2017. *Fundamentals of database systems* [in eng]. Seventh edition, global edition. Always learning. Harlow: Pearson Education Limited. ISBN: 1292097612.
- Foo, Michelle Xiao-Lin, Luca Della Libera, Ilhan Aslan, Johanne Trippas, Jeff Dalton, Cosmin Munteanu, Mary Ellen Foster, and Martin Halvey. 2022. “Paprr Readr Bot: A Conversational Agent to Read Research Papers” [in eng]. In *Proceedings of the 4th Conference on Conversational User Interfaces*, 1–4. New York, NY, USA: ACM. ISBN: 9781450397391.
- Geeksforgeeks.com. n.d. *Difference Between GUI and CUI?* <https://www.geeksforgeeks.org/operating-systems/what-is-the-difference-between-gui-and-cui/>. Last visited: 15-12-2025.
- Google. n.d. *Using OAuth 2.0 to Access Google APIs*. <https://developers.google.com/identity/protocols/oauth2>. Last visited: 17-12-2025.
- Gurol-Urganci, Ipek, Thyra de Jongh, Vlasta Vodopivec-Jamsek, Rifat Atun, and Josip Car. 2013. “Mobile phone messaging reminders for attendance at healthcare appointments.” *Cochrane Database of Systematic Reviews* (12). <https://doi.org/10.1002/14651858.CD007458.pub3>.
- HappyHelper. 2025. *Booking*. <https://happyhelper.dk/>. Last visited: 15-12-2025.

- Interaction Design Foundation. 2025. *Personas: What They Are*. Accessed: 2025-10-17. <https://www.interaction-design.org/literature/topics/personas>.
- Larman, Craig. 2004. “Extreme Programming (XP).” In *Agile and Iterative Development: A Manager’s Guide*. Addison-Wesley.
- Mathiassen, Lars. 2018. *Object-oriented analysis and design*. [in eng]. 2. edition. Hadsund: Metodica. ISBN: 9788797069301.
- Moran, Kate, and Kelley Morgan. 2023. *How to Conduct a Heuristic Evaluation*. <https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/>. Last visited: 08-12-2025.
- Nielsen, Jacob. 2023a. *AI Is First New UI Paradigm in 60 Years*. <https://www.uxtigers.com/post/ai-new-ui-paradigm>. Last visited: 15-12-2025.
- . 2023b. *The Articulation Barrier: Prompt-Driven AI UX Hurts Usability*. <https://www.uxtigers.com/post/ai-articulation-barrier>. Last visited: 15-12-2025.
- Nielsen, Jakob. 1994. *Severity Ratings for Usability Problems*. <https://www.nngroup.com/articles/how-to-rate-the-severity-of-usability-problems/>. Last visited: 10-12-2025.
- . 2024. *10 Usability Heuristics for User Interface Design*. <https://www.nngroup.com/articles/ten-usability-heuristics/>. Last visited: 10-12-2025.
- OpenAI. n.d. *ChatGPT*. <https://chatgpt.com/>. Last visited: 15-12-2025.
- OpenAI Platform. n.d. *Using tools*. <https://platform.openai.com/docs/guides/tools?tool-type=web-search>. Last visited: 10-12-2025.
- Patton, Jeff. 2014. *User Story Mapping: Discover the Whole Story, Build the Right Product*. O’Reilly Media. ISBN: 9781491904909.
- Pressman, Roger S. 2000. *Software engineering : a practitioner’s approach*. [in eng]. 5. ed. London: McGraw-Hill. ISBN: 0077096770.
- Schmidhuber, Johanna, Stephan Schlogl, and Christian Ploder. 2021. “Cognitive Load and Productivity Implications in Human-Chatbot Interaction.” In *2021 IEEE 2nd International Conference on Human-Machine Systems (ICHMS)*, 1–6. IEEE, September. <https://doi.org/10.1109/ichms53169.2021.9582445>. <http://dx.doi.org/10.1109/ICHMS53169.2021.9582445>.
- Schwaber, Ken, and Jeff Sutherland. 2020. *The 2020 Scrum Guide*. <https://scrumguides.org/scrum-guide.html>. Last visited 15-11-2025.
- Sommerville, Ian. 2016. *Software engineering* [in eng]. Tenth edition, Global edition. Harlow: Pearson. ISBN: 9781292096131.

Streamlit. 2023. *Streamlit Documentation*. <https://docs.streamlit.io>. Last visited: 09-12-2025.

Stryker, Cole. n.d. *Agentic AI: 4 reasons why it's the next big thing in AI research*. <https://www.ibm.com/think/insights/ai>. Last visited: 11/12/2025.

Sweller, John. 2020. "Cognitive load theory and educational technology." *Educational technology research and development* 68 (1): 1–16.

Virvou, Maria. 2023. "Artificial Intelligence and User Experience in reciprocity: Contributions and state of the art." *Intelligent Decision Technologies* 17 (1): 73–125. <https://doi.org/10.3233/IDT-230092>.

List of Figures

1.1	Scrumboard	2
2.1	Rich Picture of the initial problem described by Unit Cleaning	7
3.1	ER-diagram for database, prototype 1	12
3.2	Screenshot of the frontend of prototype 1	13
4.1	Rich picture of the system context.	20
5.1	Use case specification for "Jonas booking appointment"	23
5.2	Use case diagram of Cleaning Assistant System"	23
5.3	Architecture of the second prototype	26
5.4	ER-diagram of the database for second prototype	28
5.5	Calendar Widget call for authorisation	29
5.6	Calendar Widget of a given date	30
5.7	Weather forecast frontend	31
5.8	Diagram of the use of AI in the system	32
5.9	Frontend	35
5.10	Manual Calendar	36
6.1	Screenshot of an expert giving detailed information	40
6.2	Screenshot of AAI being unclear about address registration	41
6.3	Screenshot of the AI-agent forgetting the context of the conversation	42
6.4	Diagram mapping usability issues along frequency and impact on problem formulation	43

Appendix overview

Appendices are submitted as a separate document and consist of:

- Appendix A – Case Unit Cleaning, previously named Data Cleaners.pdf
- Appendix B – Case partners feedback.pdf
- Appendix C – Workbooks from Evaluation.pdf
- Appendix D - P9 Personas.pdf
- Appendix E - P9 User Stories and Use Cases Text.pdf